

Learning Software Bug Reports: A Systematic Literature Review

GUOMING LONG, University of Electronic Science and Technology of China, Chengdu, China

JINGZHI GONG, University of Leeds, Leeds, United Kingdom

HUI FANG, Loughborough University, Loughborough, United Kingdom

TAO CHEN, University of Birmingham, Birmingham, United Kingdom

The recent advancement of artificial intelligence, in particular Machine Learning (ML), has witnessed its significant growth in various software engineering research fields. Among them, bug report analysis is one of such examples as it aims to automatically understand, extract and correlate information from the reports with the help of ML approaches. Despite the importance of ML in automating and enhancing bug report analysis, a comprehensive review that systematically examines the state-of-the-art in this area is still lacking. In this article, we provide a systematic literature review on this promising research topic. Our review covers 1,825 papers, from which we extract 204 most relevant studies for detailed analysis. Based on the statistics and trends observed in these reviewed studies, we obtained seven key findings summarized as follows: (1) The extensive use of Convolutional Neural Network (CNN), Long Short-Term Memory (LSTM) and k -Nearest Neighbor (k NN) for bug report analysis, noting the underutilization of more advanced models like BERT due to their complexity and computational demands. (2) Word2Vec and TF-IDF are the most common methods for feature representation, with a notable increase in deep learning-based methods in recent years. (3) Stop word removal is the most common preprocessing method, followed by tokenization and stemming. Structural methods surged post-2020. (4) Eclipse and Mozilla are the most frequently evaluated software projects, reflecting their prominence in the field. (5) Bug categorization is the most popular task, followed by bug localization, assignment, and severity/priority prediction, with a growing interest in bug report summarization driven by advancements in NLP. (6) Most studies focus on general bug types, but there is increasing attention on specific bugs such as non-functional and performance bugs. (7) Common evaluation metrics include F1-score, Recall, Precision, and Accuracy, but bug report related evaluation metrics have not received significant attention. The majority of studies prefer k -fold cross-validation for model evaluation. (8) Many studies lack robust statistical tests or effect size measurements. Finally, based on the key findings, we discover six promising future research directions, by which we hope, together with the findings, can offer useful insights to practitioners of this particular research direction.

CCS Concepts: • **Software and its engineering** → **Maintaining software**;

Guoming Long is also supervised in the IDEAS Lab.

This work was supported by an NSFC Grant (62372084) and a UKRI Grant (10054084).

Authors' Contact Information: Guoming Long, University of Electronic Science and Technology of China, Chengdu, China; e-mail: logan1234456@outlook.com; Jingzhi Gong, University of Leeds, Leeds, United Kingdom; e-mail: j.gong@leeds.ac.uk; Hui Fang, Loughborough University, Loughborough, United Kingdom; e-mail: h.fang@lboro.ac.uk; Tao Chen (corresponding author), University of Birmingham, Birmingham, United Kingdom; e-mail: t.chen@bham.ac.uk.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).

ACM 1557-7392/2026/6-ART193

<https://doi.org/10.1145/3750040>

Additional Key Words and Phrases: Bug Report Analysis, Machine Learning, Natural Language Processing, Software Maintenance

ACM Reference format:

Guoming Long, Jingzhi Gong, Hui Fang, and Tao Chen. 2026. Learning Software Bug Reports: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 35, 7, Article 193 (June 2026), 47 pages.

<https://doi.org/10.1145/3750040>

1 Introduction

Bugs are prevalent in the process of software development, implementation, and maintenance. These software bugs, if left untreated, can often cause serious consequences. For example, Knight Capital Group, one of the biggest American market makers, lost 440 million dollars in just 30 minutes due to a software bug.¹ A common practice for bug fixing is to leverage bug reports, which document software issues encountered by developers and users. However, as the complexity and scale of software systems increase, handling bug reports becomes profoundly complex and time-consuming. As such, automated bug report analysis (or simply bug report analysis) is a vital way to free developers from tedious manual and repetitive tasks involved, achieving more efficient and effective bug fixing during software maintenance.

During the past decade, automatic analysis of bug reports has been an effective way to prioritize, track, and understand bugs, improving bug-fixing efficiency and productivity [106, 190]. However, it is a challenging task to achieve such automation due to the following two reasons: (1) The sheer volume of bug reports continues to grow exponentially. For example, in addition to the unresolved ones, the number of newly reported issues for the PyTorch project in 2023 has reached 8,352,² which is a 44× increment from what was reported in 2016. (2) Bug reports are inherently unstructured, consisting of free-form text written by various reporters with different levels of detail and technical expertise.

Consequently, in the last decade, there has been an increasing trend to exploit **Machine Learning (ML)** algorithms for automatic bug report analysis. ML has proven to be a crucial tool in this domain by enabling automated classification, deduplication, prioritization, and summarization of bug reports. Traditional rule-based approaches struggle to handle the complexity and scale of modern bug reports, whereas ML methods can learn patterns from historical data and make informed decisions with minimal human intervention. Moreover, recent advances in **Deep Learning (DL)** and **Natural Language Processing (NLP)** have further boosted the performance of bug report analysis, making ML an indispensable approach for modern software maintenance. For example, **Long Short-Term Memory (LSTM)** networks have been effectively applied to bug report analysis, capturing contextual dependencies and retaining long-term information to improve tasks such as bug report classification and summarization [100]. Further, Transformer-based models like **Bidirectional Encoder Representations from Transformers (BERT)** have demonstrated strong performance in bug report prioritization, leveraging contextual information to rank reports based on severity and urgency, enabling faster issue resolution [194].

1.1 Motivation

Despite the rapid evolution of ML methods and their growing application in software maintenance, the literature remains fragmented—many studies concentrate on specific tasks, such as duplicate

¹<https://raygun.com/blog/costly-software-errors-history/>.

²<https://tinyurl.com/294mrax5>.

detection [71, 105] or triaging [12, 112], without providing a comprehensive view of how these techniques connect and complement each other.

Moreover, while several reviews exist in adjacent areas of **Software Engineering (SE)**, few have specifically addressed the integration of ML in bug report analysis [49, 143, 149], and those that do are often outdated given the swift progress in DL and NLP [139, 190]. This lack of a unified, up-to-date synthesis hampers both researchers and practitioners, who need to understand the current state of the art, identify recurring challenges, and uncover emerging trends.

A systematic survey in this domain is therefore essential to bridge the gap between isolated findings and to provide a comprehensive roadmap that can guide future research and practical tool development. First, this survey is anticipated to offer fresh insights by formulating a unified taxonomy that integrates traditional ML techniques with recent advancements in DL and NLP. Readers will be introduced to emerging trends—such as the increasing adoption of DL algorithms, innovative feature representation methods, and evolving preprocessing strategies—as well as to critical research gaps like the underutilization of bug report-specific evaluation metrics and the lack of rigorous statistical testing, thereby fostering innovation and direct future investigations toward more effective, data-driven approaches to bug report analysis.

Beyond academic contributions, this work also seeks to support practical advancements in SE. For example, the integrative perspective provided by this survey will serve as a guide to explore underinvestigated areas and refine existing methodologies, ultimately advancing the state of the art in automated bug report analysis. Additionally, the synthesis of best practices and emerging trends will offer actionable strategies for enhancing bug-fixing processes, reducing maintenance efforts, and improving overall software quality.

1.2 Contributions

Driven by these motivations, we conduct a **Systematic Literature Review (SLR)** following the widely used SLR methodology in SE [89]. Specifically, we reviewed 1,825 papers from 46 venues and three repositories, from which we extracted 585 highly relevant studies for detailed analysis. In a nutshell, our key contributions can be summarized as follows:

1.2.1 New Insights and Findings. Our study presents a novel taxonomy that bridges traditional ML techniques with modern DL and NLP approaches for bug report analysis, providing a cohesive view of the field's evolution. In doing so, we uncover several important findings and insights, for example:

- *Emergence of DL:* The following findings indicate a shift from classical methods to DL, which reflects a growing sophistication in handling textual data for bug report analysis.
- *Finding 1: Convolutional Neural Networks (CNNs), LSTM, and k -Nearest Neighbor (kNN)* have become the leading algorithms, with a marked increase in the use of DL techniques from 2020 to 2023.
- *Finding 2:* While traditional methods like Word2Vec and **Term Frequency-Inverse Document Frequency (TF-IDF)** remain popular, there is a clear trend towards adopting DL-based feature representations.
- *Finding 3:* Standard preprocessing techniques such as stop word removal, tokenization, and stemming continue to be widely applied, accompanied by a notable rise in the use of structural preprocessing methods.
- *Attention on Evaluation Datasets and Bug Task Analysis:* Our findings reveal a dual focus: reliance on well-established datasets and an emerging interest in diversifying the types of bug-related tasks addressed. This indicates a maturing field that is beginning to tackle more complex and varied challenges.

- *Finding 4:* Benchmark projects like Eclipse and Mozilla Core are frequently used for evaluation, with a growing emphasis on projects that include unstructured bug reports.
- *Finding 5:* Bug categorization remains the dominant task, yet bug report summarization has seen significant growth, expanding from one primary study to nine over the past 4 years.
- *Finding 6:* Although most studies focus on general bug analysis, there is an increasing trend towards examining specific bug types, reflecting a deeper inquiry into nuanced software issues.
- *Gaps in Methodology and Evaluation Practices:* We also uncover limitations in the evaluation frameworks, particularly the underutilization of bug report-specific metrics and rigorous statistical methods. This gap highlights opportunities for refining the analytical rigor in future studies.
- *Finding 7:* Conventional metrics such as precision, F1-score, accuracy, and recall are predominantly used, with most studies relying on k -fold **Cross-Validation (CV)**, while bug report-specific evaluation measures remain scarce.
- *Finding 8:* Although statistical tests like the Wilcoxon signed-rank test and Cliff's delta are sometimes applied, many studies overlook rigorous statistical validation and fail to report effect sizes.

1.2.2 Impacts on the SE Community. The insights and recommendations distilled from this survey are expected to make a significant impact on the SE community—(1) for researchers, our findings serve as a guide to identify promising directions and methodological improvements for advancing automated bug report analysis; and (2) for practitioners, the synthesized best practices and emerging trends offer actionable strategies to enhance automated bug-fixing processes, thereby reducing maintenance efforts and improving software quality.

In addition to these immediate impacts, our work outlines several actionable future research directions that can further advance the field. Specifically, we propose that future work should:

- Leverage Transformer-based architectures and **Large Language Models (LLMs)** (e.g., BERT, GPT-4) to improve the precision and efficiency of tasks such as bug triaging, duplicate detection, and report summarization.
- Develop advanced, integrated tools for automated bug report analysis on platforms like GitHub by combining traditional ML, DL, and hybrid methods to address scalability and contextual challenges.
- Investigate specialized approaches for analyzing specific types of bugs by developing domain-specific models and dedicated evaluation metrics tailored to capture the unique characteristics of bug reports.
- Incorporate explainable DL techniques, such as attention mechanisms and hybrid models, to provide transparent and actionable insights, fostering stronger collaboration between researchers and practitioners.

By consolidating disparate studies into a unified framework, this work not only advances theoretical understanding but also paves the way for more effective, data-driven solutions in real-world software maintenance scenarios.

1.3 Organization

This article is organized as follows: Section 2 provides background information on bug report analysis. Following it, Section 3 presents the SLR protocol. Section 4 analyzes the results of the **Research Questions (RQs)**. Subsequently, future research directions are discussed in Section 5. Finally, the threats to validity is discussed in Section 6, and conclusions are drawn in Section 7, respectively.

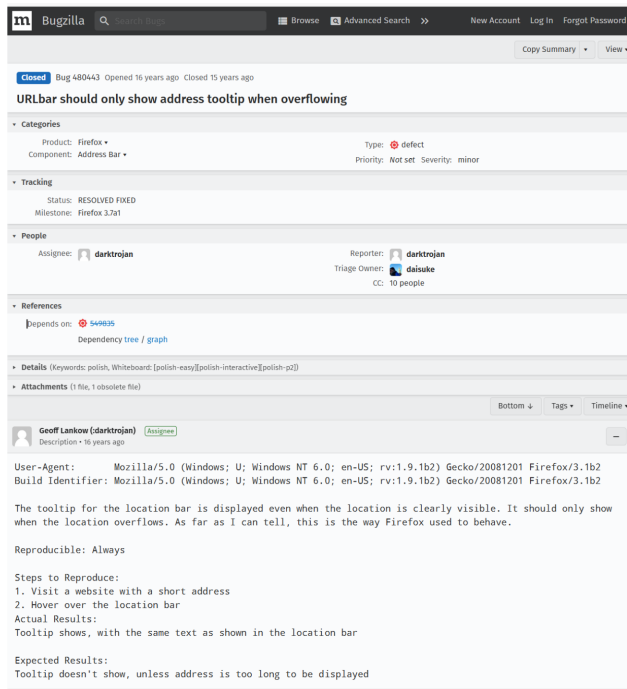


Fig. 1. A bug report from the Mozilla project.

2 Background

2.1 Bug Report

Software bugs refer to the defects in the code or the software configurations that generate unexpected behaviors [190]. An example of a Mozilla bug report³ has been shown in Figure 1. As can be seen, a typical bug report contains various attributes and fields, among which the most common ones include:

- *Title*: A short summary of a bug report.
- *Priority and Severity*: Indication of the importance and severity level of the bug.
- *Status*: Stage of the bug in the lifecycle of bug fixing, e.g., *closed*, *resolved* and *reopened*.
- *Assignee*: The person responsible for fixing the bug.
- *Description*: Detailed textual information, such as what the bug is, its affected modules, and steps to reproduce.

A bug report is produced by either a developer, a tester, or an end-user describing their identified bug, its stack trace, and the steps to reproduce it, and so on [149]. Therefore, bug reports provide useful information for the developers to fix bugs. Large-scale software projects, such as Mozilla, Eclipse, and Netbeans, have their own repositories to store bug reports. Figure 2 illustrates how the bug reports are processed in a standard procedure.

³https://bugzilla.mozilla.org/show_bug.cgi?id=480443.

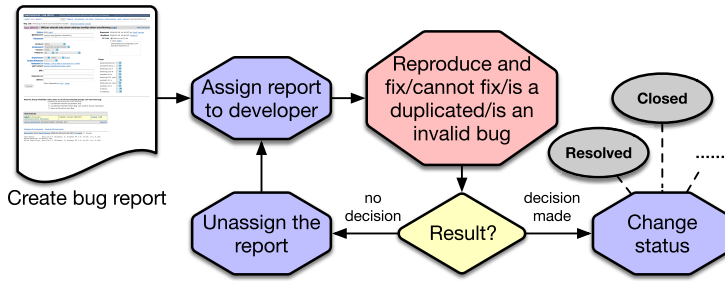


Fig. 2. Typical bug report handling protocol.

2.2 ML Algorithms

The application of ML algorithms has surged with the advancement of computational capabilities. Fundamental paradigms like statistical learning and DL are at the core of this technological evolution:

Statistical Learning methods involve models that infer patterns from data based on statistical principles. In the context of bug report analysis, statistical learning can utilize features such as the frequency of specific keywords, the length of the bug report, the time of submission, the severity of the bug, and the historical data of similar bugs. These attributes help in identifying patterns and making predictions about new bug reports.

DL techniques, employing complex neural networks, are particularly adept at understanding patterns in large volumes of data. These techniques excel in tasks that require the analysis of unstructured data, such as the textual content of bug reports. By automatically extracting relevant context features from the bug report, DL can enhance the accuracy of both discriminative and generative models. LLMs are a transformative subfield within DL, leveraging transformer-based architectures and self-attention mechanisms to efficiently process sequential data at significant scale. LLMs can be categorized into various architectural types, including encoder-only, decoder-only, and encoder-decoder models. Encoder-only models specialize in understanding contextual relationships within texts but are typically not classified as LLMs due to their smaller size and limited generative capabilities. In contrast, decoder-only models, like OpenAI's GPT-4⁴ (closed-source) and Meta's open source LLaMA 3,⁵ are optimized for generating coherent and contextually relevant text. Unlike traditional neural networks, LLMs undergo extensive pre-training on vast and diverse textual corpora, allowing them to capture intricate linguistic patterns, contextual dependencies, and semantic relationships. These models demonstrate superior capabilities in natural language understanding and generation, making them highly effective across a wide array of complex text-based applications.

2.3 Related Surveys

Despite the growing body of work on bug report analysis, to the best of our knowledge, there is no comprehensive, up-to-date review focusing exclusively on ML-based approaches. One of the earliest surveys related to bug report analysis was conducted by Strate and Laplante [139], covering 104 studies. That survey reviews the connections among various domains related to bugs/defects, such as bug report analysis, bug detection, and bug repair, rather than solely concentrating on bug report analysis. Similarly, in 2015, Zhang et al. [190] reviewed work specifically on analyzing bug reports, including papers published before or in 2014. Their focus has been on providing a

⁴<https://openai.com/index/gpt-4/>.

⁵<https://llama.meta.com/llama3/>.

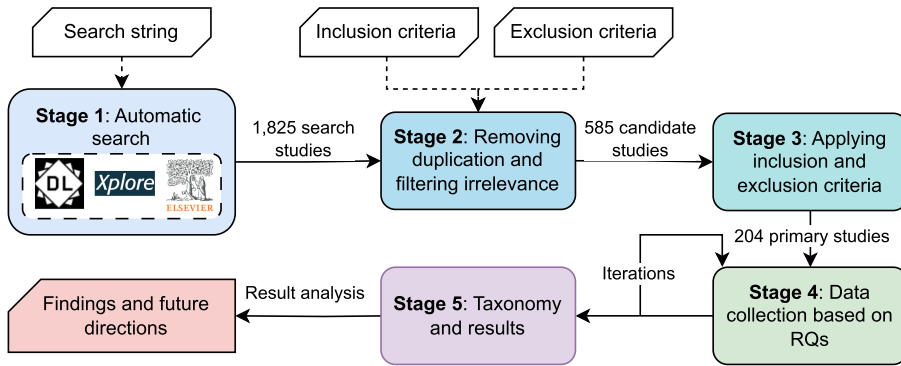


Fig. 3. Overview of the SLR protocol.

comprehensive overview of various tasks by using bug report analysis. Interestingly, the above reviews have not applied a systematic protocol as we use in this work.

There also exist surveys on specific problems in bug report analysis, e.g., bug prioritization [149], bug report summarization [143], and severity prediction [49], as opposed to the entire bug report analysis domain. A more recent survey on bug report analysis was conducted by Zou et al. [199]. Their purpose is to understand whether the problems in bug report analysis are important from the perspective of industrial practitioners. As part of the process, they carried out a systematic review on 121 studies with the sole aim of identifying all the problems that are being investigated. The result of that review forms the basis for designing a questionnaire for developers to rank their importance.

While prior surveys have examined bug reports across different platforms, they have largely overlooked the differences between desktop software and mobile applications. Zhang et al. [187] systematically analyzed bug reports in GitHub and found that mobile app reports tend to be shorter but contain more debugging elements, such as stack traces and code examples, which contribute to faster bug resolution. In contrast, desktop software reports are longer yet often lack these crucial elements, leading to longer fixing times. This distinction highlights the importance of tailored approaches in bug report analysis based on software type.

In comparison to other surveys, we seek to systematically understand what, how, why, and where ML algorithms are used and assessed in various tasks related to bug report analysis. Thus our work is in fact complementary to the work by Zou et al. [199], as well as to those problem-specific surveys [49, 143, 149]. Although ML-based methods have been briefly discussed by Strate and Laplante [139] and Zhang et al. [190], our work provides a more comprehensive review of this specialized topic, i.e., bug report analysis, by summarizing key findings and capturing the latest trends, particularly on their designed algorithms, feature representations, and evaluations, which have not been covered in previous surveys.

3 Review Methodology

In this article, we conduct an incremental SLR on bug report analysis with ML, for which most of the processes follow the best practice on SLR for SE [89].

3.1 Stage 1: Automatic Search

As shown in Figure 3, starting from *Stage 1*, from 18 January to 27 December 2023, we conducted an automatic search over three well-known indexing services, i.e., ACM Library, IEEE Xplore, and Google Scholar from 2014 to 2023 by using the search string below:

("bug report" OR "issue report" OR "fault report") AND ("machine learning" OR "artificial intelligence" OR "data mining" OR "information retrieval")

This manual search is enhanced with a focused automated search to increase the reliability of the findings obtained manually and to enhance reproducibility. Even though manual searches require significant effort, they often yield more comprehensive results compared to automated searches. Automated searches utilizing standard search strings on indexing services may overlook relevant literature, especially those addressing interdisciplinary topics that lack established terminology. Moreover, focused searches conducted at specific reputable venues can ensure the retrieval quality.

3.2 Stage 2: Removing Duplication and Filtering Irrelevant Studies

In the second stage, our goal is to ensure that the reviewed studies are both unique and highly pertinent. To this end, we remove duplicates and carefully check if the articles are relevant. This step guarantees that every study we include offers distinct contributions to the knowledge. Following this, we perform a preliminary assessment to exclude studies that are clearly not related to bug report analysis. Through these filtering steps, we have selected 585 candidate studies that are highly relevant to our investigation.

3.3 Stage 3: Applying Inclusion and Exclusion Criteria

Inclusion and exclusion criteria help us to identify studies directly related to our RQs. We apply inclusion and exclusion criteria during the initial search as well as the later stages to those studies identified through the initial search. A paper is selected as a primary study if it meets all of the inclusion criteria below:

- The paper pertains specifically to the software domain. Our focus is only on bug report analysis in SE. Investigations concerning hardware or alternative system types are out of the scope of this survey.
- The paper seeks to address a problem in bug report analysis using ML algorithms.
- The paper clearly states which ML algorithm(s) has been applied in its work.
- The paper discusses how to design bug report analysis solutions.
- The paper clarifies its experimental setup and presents quantitative experimental results.
- The paper performs evaluations based on real-world software projects.

For each paper selected after the initial search, we apply the following exclusion criteria. A study is removed if it meets any of the following criteria:

- The paper type is either an editorial, abstract, opinion paper, short paper, poster abstract, keynote, opinion, summary of a tutorial, summary of a conference (or an introduction to conference proceedings), summary of a workshop, or a summary of a panel discussion.
- The paper is not SE related.
- The paper is not published in a peer-reviewed venue.

The above process stops when we finish reviewing all candidate studies. Finally, 204 primary studies are selected for further data collection.

3.4 Stage 4: Data Collection Based on RQs

In this stage, we formulate the RQs for this literature review outlined below. Precisely, we have identified six questions suitable for investigation.

3.4.1 RQs. Initially, considering the potential utilization of diverse ML algorithms in bug report analysis, the primary area involves exploring ML algorithms, feature representation and preprocessing techniques. In order to grasp this concept, we pose the first RQ:

RQ1: What ML algorithms, feature representation and preprocessing methods are used?

Subsequently, we observed that the bug reports used in bug report analysis as datasets are sourced from a multitude of software projects. In order to investigate these software projects, we aim to comprehend:

RQ2: What software projects are evaluated?

Furthermore, we explore the multifaceted landscape of the associated tasks of bug report analysis using ML techniques. Our aim is to explain the detailed challenges and issues encountered within this domain. Through a comprehensive examination, we seek to clarify:

RQ3: What tasks in bug report analysis are tackled by ML?

Moreover, an integral aspect of our investigation relates to the categorization of bug types within the selected studies. To understand the breadth and scope of bug types used in bug report analysis, we pose the question:

RQ4: What types of bugs are analyzed?

Additionally, to measure the effectiveness and reliability of bug report analysis techniques, we carefully examine the evaluation metrics and methodologies employed across various studies. Hence, we inquire:

RQ5: Which evaluation metrics and procedures are leveraged in the evaluation?

Lastly, to ensure the robustness and validity of collected bug report analysis studies, we investigate the statistical tests and standardized effect size measurements utilized by researchers to mitigate potential biases in bug report analysis studies. Therefore, we ask:

RQ6: Which statistical tests and standardized effect size measurements are applied to mitigate bias?

3.4.2 Data Items. We have devised an extensive list of data items during the review process to answer the RQs. The data elements, listed in Table 1, comprise a total of 12 items, each serving a specific purpose pertaining to its corresponding RQ. In this section, we explain the rationale behind their design and explain the procedure for extracting and categorizing data from these items. As shown in Table 1, the data items F1 to F3, including primary study ID, title, author, and publisher, reflect the meta-information of the reviewed studies. This meta-information facilitates accurate referencing and organization of the studies, laying a solid groundwork for subsequent analysis and interpretation.

To address RQ1, we aim to explore ML algorithms, feature representations and preprocessing methods used in these primary studies. For this purpose, data items F4 to F6 are crafted to extract

Table 1. Data Items for Collection

Field ID #	Field	RQ
F1	Primary ID	N/A
F2	Title	N/A
F3	Publication site	N/A
F4	Algorithm	RQ1
F5	Representation	RQ1
F6	Preprocessing	RQ1
F7	Project	RQ2
F8	Task	RQ3
F9	Bug type	RQ4
F10	Metrics	RQ5
F11	Procedure	RQ5
F12	Statistical test	RQ6
F13	Effect size	RQ6

information on these methods. For RQ2, we use data item F7 to investigate which software projects are evaluated in their bug report analysis. We examine F8 to explore what particular tasks are tackled by using ML in RQ3. After it, we explore what types of bugs are analyzed by examining F9 for RQ4. Furthermore, we discuss the evaluation metrics and procedures leveraged in the evaluation using F10 and F11 for RQ5, respectively. Finally, to answer RQ6, F12 is designed to identify which statistical tests are used in the studies, whereas F13 is for further clarifying the effect size measurement that examines the magnitude of difference in the statistical tests.

3.4.3 Data Collection. The process of data collection aims to obtain thorough information from the primary studies, as outlined in Table 1. The collection protocol follows standard practices commonly employed in SE literature reviews [89]. This involves a series of steps conducted by all authors in three iterations.

During the initial iteration, each author independently conducts data collection from the 204 primary studies. They meticulously review each study, extract data according to predefined criteria, summarize it into a table with 12 columns and 204 rows, and conduct initial classification. Some challenges arise where certain data items are missing, poorly defined, or require inference from context. Such instances are flagged for further investigation in subsequent iterations.

In the second iteration, authors cross-reference and merge their individual data summaries into an integrated table. Unclear data items are revisited and resolved through discussions, further research, or consultation with external experts. For instance, a category labeled “unspecified” is added for the evaluation procedure due to a lack of justification in many studies, requiring additional research to determine the subject system domains.

The third iteration aims to summarize the integrated table statistics, count the number of studies for each technique associated with data items, and group similar techniques into broader categories. For example, studies focusing on bug report title generation are grouped under “bug report summarization” due to shared characteristics. All authors participate in comprehensive discussions to finalize data classifications for the study.

Finally, this process has resulted in the data used in this study,⁶ which undergoes thorough discussion among all authors to achieve comprehensive classifications.

⁶Details of the primary studies and data can be found at our repository: <https://tinyurl.com/322j4eak>.

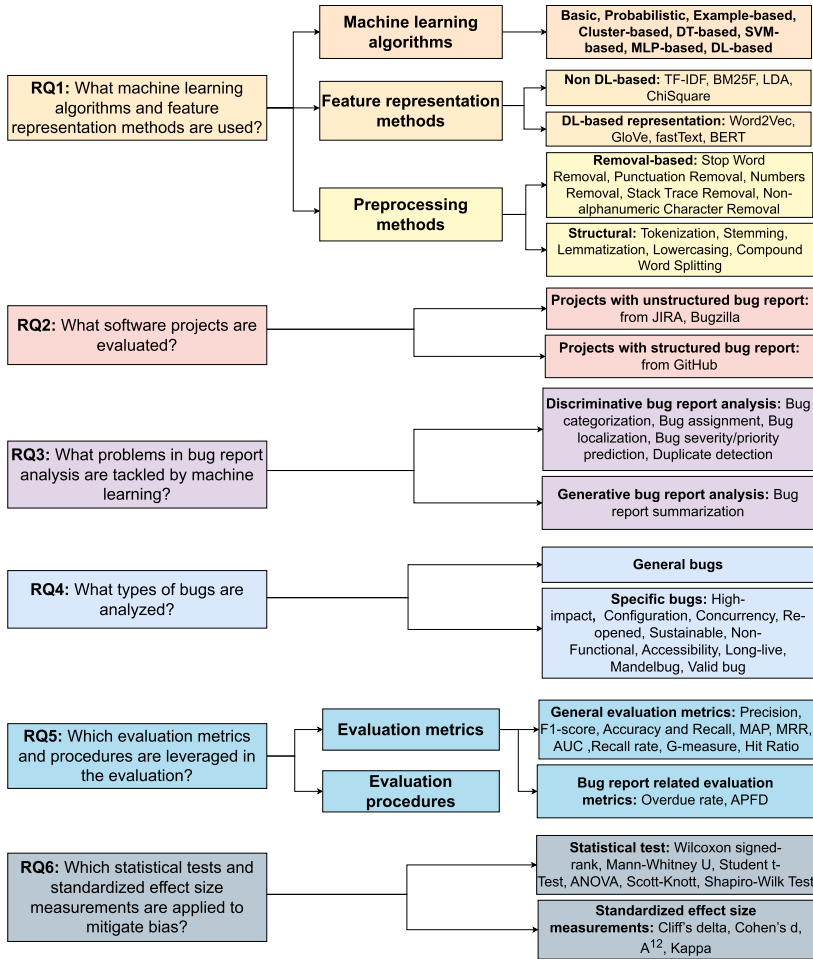


Fig. 4. The taxonomy of learning-based bug report analysis. We use color to indicate different sections.

3.5 Stage 5: The Taxonomy and Result

Using the data collected in the previous stage, we develop a taxonomy and obtain results that enhance our understanding of bug report analysis using ML techniques.

As illustrated in Figure 4, we construct a taxonomy of bug report analysis with ML techniques by connecting RQs and their corresponding data items. The main content flow and categorization of the survey are effectively summarized, focusing on various aspects of bug report analysis using ML techniques. The key elements are broken down into distinct sections, each addressing a specific area of interest. Here's what each category specifically means for the RQs:

– *RQ1: What ML algorithms, feature representation and preprocessing methods are used?*

– *ML Algorithms:* This involves various types of algorithms used for bug report analysis, including basic algorithms, probabilistic methods, example-based techniques, **Decision Tree (DT)**-based methods, **Support Vector Machines (SVMs)**, and neural network-based approaches.

- *Feature Representation Methods*: These are divided into non-DL and DL-based methods, suggesting an exploration of how bug report data is structured and utilized by different ML algorithms.
- *Preprocessing Methods*: These techniques are essential for refining raw bug report data before applying models. They can be broadly categorized into removal-based methods, which eliminate unnecessary elements, and structural methods, which transform text while preserving all information. The selection of preprocessing methods plays a crucial role in improving data quality and model performance.
- *RQ2: What software projects are evaluated?*
 - *Projects with Unstructured Bug Report*: This focuses on studies that deal with projects where bug reports are not formatted according to a predefined structure.
 - *Projects with Structured Bug Report*: This focuses on studies involving projects that use a specific, predefined format for bug reports.
- *RQ3: What problems in bug report analysis are tackled by ML?*
 - *Discriminative Bug Report Analysis*: ML techniques that classify or differentiate between types or qualities of bug reports.
 - *Generative Bug Report Analysis*: ML techniques that generate new content or predictions related to bug reports.
- *RQ4: What types of bugs are analyzed?*
 - *General Bugs*: Studies that focus on a wide range of bugs without specific limitations or definitions.
 - *Specific Bugs*: Studies that focus specifically on certain types of bugs, possibly those that are more critical or common in certain contexts.
- *RQ5: Which evaluation metrics and procedures are leveraged in the evaluation?*
 - *General Evaluation Metrics*: Metrics that are widely used across various studies and contexts.
 - *Bug Report Related Evaluation Metrics*: Specific metrics that are tailored to evaluate bug report analyses.
 - *Evaluation Procedures*: Methods and protocols used to assess the effectiveness of ML techniques in analyzing bug reports.
- *RQ6: Which statistical tests and standardized effect size measurements are applied to mitigate bias?*
 - *Statistical Test*: Refers to the statistical methods used to analyze data and test hypotheses in the studies.
 - *Standardized Effect Size Measurements*: Measures used to quantify the size of an effect or difference seen in the studies, helping to address the magnitude of impacts in statistical terms.

In the next section, we will present the results analysis, where a thorough examination of the comprehensive data statistics gathered from the primary studies will be discussed. This analysis will reveal key findings and outline potential future directions for research.

4 Results Analysis

4.1 RQ1: What ML Algorithms, Feature Representation and Preprocessing Methods Are Used?

4.1.1 ML Algorithm. Since our focus is on bug report analysis with ML, it is essential to understand what ML algorithms are of the most interest. In this section, we particularly review the ML algorithms that are commonly utilized for bug report analysis within the community. Figure 5(a) shows the Top 10 ML algorithms used in primary studies. It is observed that CNN, LSTM, and k NN

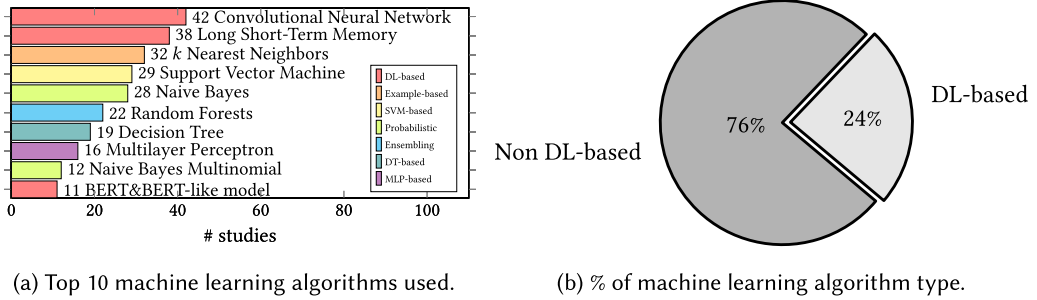


Fig. 5. Statistics of ML algorithms.

are the most commonly selected methods compared with the others. Here, we categorize the used ML methods into: basic methods, probabilistic methods, example-based methods, ensemble methods, cluster-based methods, DT-based methods, SVM-based methods, and neural network methods.

Between 2020 and 2023, there has been a notable surge in the adoption of DL-based models for bug report analysis, with a total of 58 studies utilizing these advanced computational techniques, compared to just 18 studies between 2014 and 2019. This significant increase underscores the growing recognition and reliance on DL methodologies within this domain. The enhanced capabilities of DL models to handle complex patterns and large datasets effectively have made them increasingly favored for analyzing and extracting meaningful insights from bug reports.

Here, we go through each type of ML in detail:

Basic Methods. *Linear Regression*, modeling the relationship between a dependent variable and its independent variables, has been applied to predict severity or priority levels in bug report analysis [145, 151]. However, its effectiveness may be limited by the linear assumption, whereas bug report analysis involves complex and non-linear patterns. In contrast, *Logistic Regression*, estimating probabilities with a logistic function for binary classification task, is useful in bug report identification [124, 160] and duplicate bug report detection [2, 61]. It can also be extended to multi-class tasks using a one-vs-all strategy for severity [92, 140] and priority level prediction [65, 123, 140].

Probabilistic Methods. Probabilistic classifiers predict the probability that a document belongs to a class using Bayes' theorem [96, 143]. *Naive Bayes* [129], which assumes independence between features, is used in bug report analysis due to its simplicity and effectiveness with textual data [45, 61, 124, 151, 175]. However, its assumption of feature independence often does not hold true in real-world text data, where word occurrences can be highly dependent on each other.

Example-Based Methods. *kNN* is a simple, non-parametric lazy learning algorithm for classification and regression tasks that identifies the top k objects similar to a given instance. In bug report analysis, *kNN* efficiently handles feature-rich datasets by measuring the proximity of bug reports in feature space, aiding in pattern and similarity identification [65, 140, 168, 180]. However, *kNN* can be computationally intensive, as computing distances between a query point and all training examples becomes inefficient with larger datasets.

Cluster-Based Methods. Clustering techniques group similar bug reports to identify common patterns and improve resolution strategies. Two commonly used models are: *k-means* selects centroids for each cluster and associates each feature vector with the closest centroid. It is known for its simple implementation and computational efficiency, making it effective for categorizing large volumes of bug reports [144, 154]. However, it faces challenges like determining the optimal

number of clusters and sensitivity to initial centroids and outliers. *Fuzzy c-means* allows each data point to have a degree of membership in each cluster, addressing *k-means*' limitations in handling datasets with unclear cluster boundaries [75].

DT-Based Methods. DTs use a tree-like model to make decisions by segmenting reports based on their attributes. They are effective for extracting rules and patterns from bug reports, aiding in various analysis tasks [4, 27, 46, 55, 70, 197]. DTs are noted for their interpretability and ability to handle non-linear relationships. However, they can overfit by creating overly complex trees and struggle with imbalanced data, potentially biasing the algorithm towards predominant classes [4].

Ensembling Methods. Ensemble Learning combines multiple classifiers to improve performance, accuracy, and robustness. Common methods include *voting* [163] and *bagging* [82, 95]. *Random Forest*, a bagging method, constructs multiple DTs and averages their predictions, reducing overfitting. However, bagging can struggle with imbalanced datasets, addressed by techniques like constructing classifiers on disjoint subsets [169]. *Boosting* trains models sequentially, correcting errors of previous models, with algorithms like AdaBoost [42], Gradient Boosting [43], and XGBoost [17]. Boosting is robust against unbalanced data but can be sensitive to noise and computationally demanding. *Stacking* combines model predictions using another model to find optimal weights, potentially outperforming bagging but at a higher computational cost [76].

SVM-Based Methods. SVM classifies data by finding the hyperplane that best separates classes with maximum margin. In bug report analysis, SVM is favored for handling complex, high-dimensional textual data, making it effective for various tasks [39, 63, 135, 148]. Despite its strengths, SVM can be time-consuming and sensitive to data scale and dimensionality, requiring dimensionality reduction or feature selection to maintain accuracy and efficiency [185].

Multilayer Perceptron (MLP)-Based Methods. MLP is a feedforward neural network with an input layer, hidden layers, and an output layer, interconnected through weights adjusted via the backpropagation algorithm. This process helps minimize the difference between actual and predicted outputs and addresses overfitting through regularization. Studies often compare MLPs with other algorithms, but MLPs rarely outperform alternatives like SVM and Naive Bayes in bug report analysis [47, 70, 90, 92, 123]. Despite their complexity and lengthy training times, MLPs do not consistently show superior performance.

DL-Based Methods. DL refers to a subset of ML that employs neural networks with many layers to model complex patterns in data. Unlike traditional ML algorithms that require manual feature extraction, DL methods automatically learn hierarchical feature representations, making them particularly powerful for tasks involving large and complex datasets. It's important to distinguish between DL and simpler neural network architectures like MLPs. MLPs are a type of neural network with one or more layers of nodes, typically used for tasks requiring fixed-size input and output. However, MLPs are not considered part of DL in this work because they lack the depth and complexity of architectures, which can automatically learn and extract features from data at multiple levels of abstraction.

CNNs learn spatial hierarchies of features from input data, originally designed for images but also effective for text processing in bug report analysis. They automatically extract features from text represented as numerical vectors, identifying patterns like n-grams without extensive preprocessing. This makes CNNs suitable for various bug report analysis tasks [6, 59, 153]. However, while CNNs excel in spatial feature extraction, they are less effective than **Recurrent Neural Networks (RNNs)** for tasks requiring long-term temporal dependency processing, such as time-series analysis or NLP.

RNNs, including *LSTM* and **Gated Recurrent Unit (GRU)** architectures, are essential in bug report analysis for processing sequential data and retaining past information. RNNs capture contextual dependencies in bug reports, enhancing understanding of issues. Traditional RNNs face vanishing and exploding gradient problems, but LSTM and GRU networks address these by retaining long-term dependencies. LSTM networks are notable for their gated mechanism that maintains effective learning over long sequences, making them invaluable for bug report analysis tasks [84, 97, 100, 112, 182]. GRUs balance effectiveness and computational efficiency by simplifying gating mechanisms, suitable for large-scale bug report analysis [166]. However, GRUs' simplified structure can struggle with capturing complex dependencies.

Transformer models, such as *BERT* and its variants, have revolutionized bug report analysis by capturing bidirectional contexts and global dependencies within textual data. They excel at tasks such as bug report summarization, sentiment analysis, and identifying similarities between bug reports and code repositories. For instance, the research identified as Zheng et al. [194] introduces an approach for predicting bug severity by integrating Transformer models with domain-specific pre-training strategies. Meanwhile, study Fukuda et al. [44] leverages the model to generate readable fault reports from diverse data, demonstrating its capability in fault detection. Additionally, Wei et al. [159] employs the Transformer to refine the extraction of security domain knowledge, enhancing keyword quality. These examples highlight the model's potential in enhancing software reliability and security through sophisticated analysis of bug reports.

Despite their effectiveness, transformer models face several adoption barriers beyond theoretical limitations. The main reasons include:

- *High Computational Requirements*: Training BERT from scratch requires massive datasets, such as the 3.3 billion words used for the original BERT model, and significant GPU resources, which are often inaccessible to many research teams. A likely solution could be leveraging transfer learning, such as pre-trained models like CodeBERT [38], fine-tuned on domain-specific datasets to significantly reduce computational demands.
- *Data Scarcity*: Labeled bug report datasets are often small and imbalanced, posing practical barriers for training transformer models effectively. A possible solution is synthetic data generation. For example, [111] employs a BERT-based encoder-decoder framework combined with a transformer decoder and copy mechanism to generate high-quality, concise bug report titles from lengthy descriptions, thus effectively mitigating data inadequacy in low-resource settings.
- *Lack of Interpretability*: The “black-box” nature of transformer-based DL models presents significant challenges, particularly for trust-sensitive tasks like severity prioritization. One way to address this challenge could be the integration of attention visualization into BERT-based systems. For instance, [62] introduces attention heatmaps within their fine-tuned model, achieving high performance (F1 score of 0.87) on real-world OSS defect reports, while transparently indicating how category-specific keywords influence classification decisions.

Discussions. Through our investigation, as the increasing trend to embrace DL methods, BERT and its variants, along with other transformer models, have been highlighted for their superior performance in bug report analysis tasks. However, within the period from 2020 to 2023, only 14 studies have employed BERT and BERT-like or transformer-based models. As illustrated in Figure 5(b), there is a modest 24% of all the DL-based approaches, which suggests that their adoption is not as widespread as the general category of DL models, although their merit for bug report analysis is highly recognized.

The relatively lower adoption rate of BERT and BERT-like and transformer-based models may be attributed to several factors, including the requirement for substantial computational resources and

Table 2. Summaries of the Strengths, Weaknesses, and Best-Suited Usage Scenarios for the ML Approaches

Category	Strength	Weakness	Best-Suited Scenario
Basic methods	Simple and interpretable models that are computationally efficient.	Limited by linear assumptions and sensitive to outliers.	Predicting severity or priority levels in bug reports with linear relationships.
Probabilistic methods	Simple and fast, effective with large datasets and handles missing data well.	Assumes feature independence, which often does not hold true in real-world data.	Text classification tasks and bug report analysis where computational efficiency is crucial.
Example-based methods	Simple and intuitive, effective with multi-modal data and no training phase required.	Computationally intensive with large datasets and sensitive to irrelevant features.	Pattern and similarity identification in feature-rich bug report datasets.
Cluster-based methods	Simple implementation, computationally efficient, and handles large datasets well.	Requires pre-specifying the number of clusters and sensitive to initial centroids and outliers.	Categorizing large volumes of bug reports with well-separated clusters.
DT-based methods	Interpretable and easy to visualize, handles non-linear relationships well.	Prone to overfitting and struggles with imbalanced data.	Extracting rules and patterns from bug reports where interpretability is crucial.
Ensembling methods	Combines multiple classifiers for improved performance, accuracy, and robustness.	Computationally intensive and less interpretable than single models.	High accuracy tasks and bug report analysis with imbalanced datasets.
SVM-based methods	Effective in high-dimensional spaces and robust to overfitting.	Computationally intensive and sensitive to the choice of kernel parameters.	Classification tasks with clear margin of separation and high-dimensional data.
MLP-based methods	Capable of learning complex patterns and relationships in data.	Requires large amounts of data and computational resources for training.	Tasks involving complex, non-linear relationships and large datasets.
CNN-based methods	Excellent for image and spatial data analysis, capturing local dependencies.	Requires large datasets and computational resources, sensitive to overfitting.	Image-based bug report analysis and tasks involving spatial data.
RNN-based methods	Effective for sequential data and capturing temporal dependencies.	Computationally intensive and prone to vanishing gradient problem.	Analyzing sequential bug report data and tasks involving time-series data.
Transformer-based methods	Handles long-range dependencies well and highly parallelizable.	Requires substantial computational resources and large datasets.	NLP tasks and analyzing complex bug report texts.

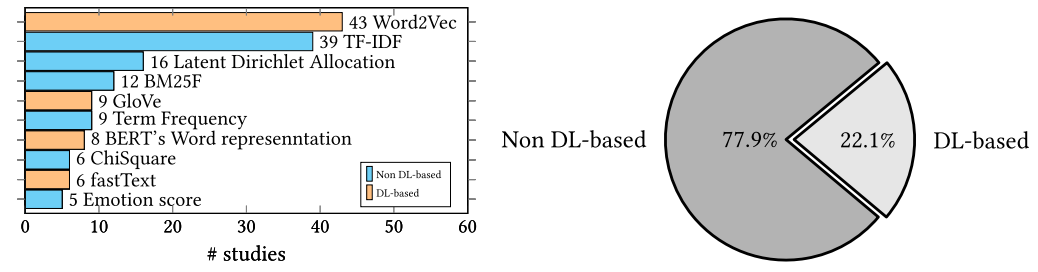
the necessity for large datasets to train these models effectively. Nevertheless, the demonstrated efficacy of these models in handling bug report analysis tasks presents a compelling case for their increased utilization, suggesting that future research and application in this area could benefit significantly from a deeper exploration and broader adoption of transformer-based DL models.

Based on our review, we have compiled Table 2 to highlight the strengths, weaknesses, and ideal use cases for various categories of ML methods. This table serves as a valuable resource for researchers and practitioners, enabling them to choose the most appropriate ML method for different scenarios.

In a nutshell, our findings can be summarized as follows:

Finding 1: CNN, LSTM and k NN are the top three commonly used ML algorithms for bug report analysis. It also notes a significant increase in the use of DL algorithms from 2020 to 2023.

4.1.2 Feature Representation. The employment of appropriate feature representation is important for extracting meaningful insights from textual data. Typical methods encompass a wide spectrum ranging from traditional non-DL approaches to the more advanced DL-based techniques. Here,



(a) Top 10 feature representation methods (TF-IDF denotes Term Frequency-Inverse Document Frequency).

(b) % of feature representation method type.

Fig. 6. Statistics of feature representation methods.

we outline the significance and utility of both categories of feature representation methods in facilitating comprehensive bug report analysis.

As illustrated in Figure 6(a), Word2Vec and TF-IDF are the most common feature representation methods, accounting for 43 and 39 studies respectively. They each have three times the number of studies compared to **Latent Dirichlet Allocation (LDA)**, which is the third most prevalent method.

Moreover, our analysis revealed an intriguing trend: compared to the period from 2014 to 2019 (with 16 studies), the years from 2020 to 2023 witnessed a substantial increase in the adoption of DL-based feature representation methods, with 48 studies.

Here, we present an overview of these methods, categorized into non-DL-based representation and DL-based representation, each serving distinct approaches in representing the intricacies of bug reports:

Non DL-Based Method. *TF-IDF* is widely used in bug report analysis to capture term importance by weighing terms based on their document frequency relative to the corpus [76, 90, 93, 145, 146]. This highlights significant terms within bug reports but ignores word order, context, and semantics, and may bias towards longer documents. Some studies use simple *TF* to count term occurrences [36, 46, 72, 99, 155, 158], offering quick insights but lacking sophistication for complex tasks.

BM25F extends BM25 with field-based weighting, making it suitable for bug report analysis where different fields have varying importance. It requires careful parameter tuning for optimal performance [61, 105, 137, 188, 189]. *LDA* uncovers latent topics within bug reports, aiding in text representation by identifying topics based on word distributions [39, 71, 77, 171, 186]. However, *LDA* struggles with large vocabularies. *Chi-Square* identifies significant terms for issue classification in bug report analysis, essential for feature selection. Higher Chi-Square values indicate stronger term-topic associations [39, 80, 136, 160, 167, 175].

DL-Based Method. *Word2Vec* captures semantic similarities between terms by representing words as dense vectors in a high-dimensional space, enhancing bug report analysis by identifying semantically similar terms [114]. Some researchers use pre-trained *Word2Vec* models, while others train their own, like the CBOW model developed by Zhang et al. [192]. However, *Word2Vec* may struggle with global corpus statistics and rare words. **Global Vectors for Word Representation (GloVe)** uses global co-occurrence statistics for deeper contextual understanding of terms in bug reports, valuable for comprehensive analysis [6, 25, 88, 107, 131]. *GloVe*, however, requires more memory and computational resources due to its large co-occurrence matrix.

fastText is used in bug report analysis [57, 79, 140, 141, 195] for its ability to incorporate sub-word details, managing out-of-vocabulary terms and morphologically complex languages. By summing character n-grams to represent words, *fastText* provides robust embeddings that handle unseen terms and capture morphological variations, enhancing the vector representations for bug summaries and keywords.

BERT's word representation, derived from BERT, is highly effective in capturing contextual information within bug reports. By considering the surrounding context of each word, BERT's embeddings provide rich representations that account for the nuanced meanings of terms within their specific contexts, yielding rich representations that encapsulate nuanced term meanings within their specific contexts. This methodology has proven to be crucial in recent bug report analysis research [58, 67, 85, 111, 113, 116, 152, 157], significantly contributing to the high accuracy achieved by the BERT model.

Discussions. We observe that Word2Vec has become the most frequently used feature representation due to the growing recognition of the importance of DL-based models, consistent with our findings in Section 4.1.2, which noted the increasing application of DL-based ML models. While DL-based ML algorithms can be combined with non-DL-based feature representation methods, such as Word2Vec with Naive Bayes or SVM, such combinations often necessitate dimensionality reduction operations like pooling, which can lead to information loss and limited improvements in accuracy. Therefore, as shown in Figure 6(b), most studies opt to combine non-DL-based ML algorithms with non-DL-based feature representation methods, or DL-based ML algorithms with DL-based feature representation methods.

In recent years, we have observed a new trend emerging in some studies, wherein researchers directly utilize the output of BERT as feature representation. We believe that future research directions should pay more attention to this approach, as BERT possesses superior capabilities in capturing contextual information and semantic nuances within textual data, potentially leading to enhanced bug report analysis performance.

Based on our review, we have created Table 3, which details the strengths, weaknesses, and optimal use cases for each feature representation method. This table serves as a practical guide for researchers and practitioners, aiding them in selecting the most appropriate feature representation method for different scenarios.

In summary, we obtain findings shown below:

Finding 2: Word2Vec is the most popular method for feature representation. TF-IDF ranked the second. There has been a notable increase in the adoption of DL-based feature representation methods from 2020 to 2023, outpacing previous years.

4.1.3 Preprocessing Method. In bug report analysis, preprocessing methods are essential for refining raw textual data to improve the effectiveness of analysis tasks. These methods help reduce noise, standardize text formats, and enhance the quality of extracted features. Figure 7(a) shows the top 10 preprocessing methods used in primary studies. It is observed that stop word removal is the most frequently used method, appearing in 138 papers. Tokenization and stemming rank second and third, with 110 and 98 papers, respectively. These three methods are dominant, as there is a significant drop starting from the fourth-ranked method, lowercasing, which appears in only 44 papers.

Preprocessing techniques can be broadly categorized based on whether they involve the removal of certain elements from the text. We classify preprocessing methods into two groups: Removal-based methods and structural methods. Removal-based methods eliminate specific components from the text, such as stop words, punctuation, or stack traces, to reduce redundancy and focus on

Table 3. Summaries of the Strengths, Weaknesses, and Best-Suited Usage Scenarios for the Feature Representation Methods

Category	Strength	Weakness	Best-Suited Scenario
TF-IDF	Captures term importance by weighing terms based on their document frequency relative to the corpus.	Ignores word order, context, and semantics; may bias towards longer documents.	Highlighting significant terms within bug reports.
TF	Simple and quick insights by counting term occurrences.	Lacks sophistication for complex tasks.	Basic analysis of term occurrences in bug reports.
BM25F	Extends BM25 with field-based weighting, suitable for varying field importance.	Requires careful parameter tuning for optimal performance.	Analyzing bug reports with different field importances.
LDA	Uncovers latent topics within bug reports by identifying topics based on word distributions.	Struggles with large vocabularies.	Topic modeling and text representation in bug reports.
Chi-Square	Identifies significant terms for issue classification, essential for feature selection.	May not capture complex term relationships.	Feature selection for issue classification in bug reports.
Word2Vec	Captures semantic similarities between terms by representing words as dense vectors.	May struggle with global corpus statistics and rare words.	Identifying semantically similar terms in bug reports.
GloVe	Uses global co-occurrence statistics for deeper contextual understanding of terms.	Requires more memory and computational resources due to its large co-occurrence matrix.	Comprehensive analysis of terms in bug reports.
fastText	Incorporates subword details, managing out-of-vocabulary terms and morphologically complex languages.	Requires substantial computational resources.	Enhancing vector representations for bug summaries and keywords.
BERT	Captures contextual information within bug reports, providing rich representations.	Requires substantial computational resources and large datasets.	High accuracy tasks in bug report analysis, such as summarization and sentiment analysis.

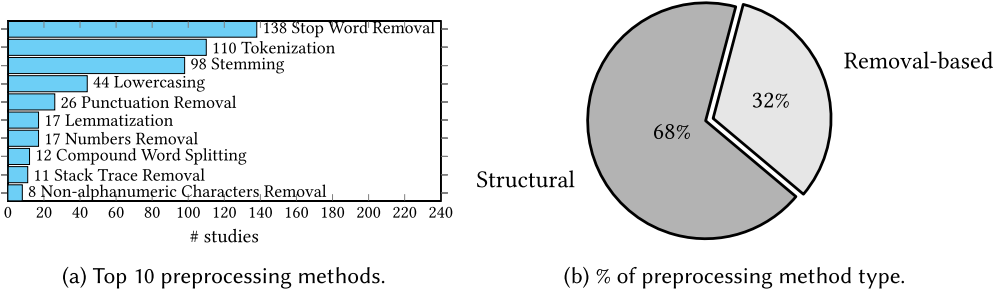


Fig. 7. Statistics of preprocessing methods.

meaningful content. In contrast, structural methods transform or normalize text without discarding any information, including techniques like tokenization, stemming, and lowercasing. As shown in Figure 7(b), our analysis reveals that 32% of the preprocessing methods used in primary studies are removal-based, while structural methods account for 68%. The following provides an overview of each category, highlighting their benefits and limitations:

Removal-Based Methods. *Stop Word Removal* [76, 90, 94, 121, 178] eliminates common but uninformative words (e.g., “the,” “is”) to reduce noise in text analysis. Its advantage is improved efficiency

and relevance in text processing, but it may lead to loss of contextual meaning. *Punctuation Removal* [8, 13, 68, 121, 124] deletes symbols such as commas and periods to standardize text input. This enhances consistency but can remove valuable syntactic cues. *Numbers Removal* [13, 68, 69, 70, 178] eliminates numerical values to prevent them from skewing textual analysis. While it reduces dimensionality, it may also discard meaningful identifiers or version numbers. *Stack Trace Removal* [48, 63, 73, 132, 133] filters out automatically generated error traces, reducing redundancy in bug reports. This helps focus on descriptive content but may remove useful debugging information. *Non-Alphanumeric Character Removal* [1, 30, 91, 124, 131] strips out special characters to simplify text representation. It increases uniformity but can result in the loss of structured information, such as code snippets or log patterns.

Structural Methods. *Tokenization* [59, 60, 97, 101, 131] splits text into individual words or subwords to facilitate further processing. It improves feature extraction but can increase data sparsity if done improperly. *Stemming* [25, 66, 87, 91, 195] reduces words to their root form to normalize variations of the same term. While it helps unify vocabulary, it may lead to over-stemming, reducing interpretability. *Lemmatization* [9, 80, 84, 126, 150] converts words to their base dictionary form, improving consistency in word representations and reducing vocabulary size. While it enhances text normalization, it requires linguistic knowledge and can be computationally expensive. *Lowercasing* [44, 125, 153, 166, 174] converts all text to lowercase to ensure uniformity in text matching. This reduces case sensitivity errors but may discard case-specific information, such as proper nouns. *Compound Word Splitting* [64, 66, 104, 137, 173] breaks down multi-word expressions into smaller components, improving granularity in text analysis. Although it enhances feature representation, it can introduce errors if not context-aware. In modern DL-based bug report analysis models, stop word removal is less necessary because these models learn contextual embeddings rather than relying on traditional frequency-based methods like TF-IDF or bag-of-words. Models such as transformers (e.g., BERT) capture semantic meaning and understand the importance of words in context, allowing them to assign appropriate weights to stop words rather than discarding them outright. Additionally, some stop words can be important for understanding the relationships between key terms in bug reports.

Based on the analysis above, we summarize the strengths, weaknesses, and best-suited scenario for each preprocessing method in Table 4. This table provides a useful resource, helping researchers and practitioners choose the most suitable methods according to their specific needs and circumstances.

Discussions. Our analysis reveals that stop word removal is the most frequently used preprocessing method. Surprisingly, from 2020 to 2023, only 27 papers employed stop word removal, accounting for just 19.6% of the total. We attribute this decline to the widespread adoption of DL-based models in recent years.

Similarly, from 2020 to 2023, 64 papers utilized tokenization, representing 58.1% of the total, compared to just 41.9% between 2014 and 2019. We believe this increase is primarily due to the reliance on modern DL-based bug report analysis models on tokenization. These models require tokenization because they rely on subword embeddings (e.g., WordPiece, Byte Pair Encoding) to handle out-of-vocabulary words, typos, and rare terms effectively. Tokenization helps break down complex words into meaningful units, ensuring that the model can learn better representations and handle diverse linguistic patterns in bug reports.

Based on these observations, we believe that future research should place greater emphasis on structural preprocessing methods. Since DL-based bug report analysis models can inherently handle stop words, out-of-vocabulary words, typos, and rare terms without needing to remove them, adopting transformation-based preprocessing techniques can further enhance bug report analysis performance.

Table 4. Summaries of the Strengths, Weaknesses, and Best-Suited Usage Scenarios for the Preprocessing Methods

Category	Strength	Weakness	Best-Suited Scenario
Stop word removal	Reduces noise and improves efficiency by removing common but uninformative words.	May lead to loss of contextual meaning.	Traditional NLP models like TF-IDF or bag-of-words where reducing dimensionality.
Tokenization	Splits text into meaningful units, facilitating feature extraction and downstream processing.	Improper tokenization may increase data sparsity and negatively impact word relationships.	Preprocessing step for word-based or subword-based models.
Stemming	Normalizes words by reducing them to their root form, improving vocabulary consistency.	Over-stemming can lead to loss of meaning and reduced readability.	Search engines or retrieval-based systems where word variations need to be matched.
Lowercasing	Ensures uniformity by converting text to lowercase, reducing case sensitivity errors.	May discard important case-sensitive information, such as proper nouns and acronyms.	Applications where case differences do not alter meaning, such as basic text classification.
Punctuation removal	Standardizes text input by removing symbols like commas and periods, improving consistency.	Removes valuable syntactic cues, which may impact grammatical analysis.	When punctuation is not crucial, such as when using bag-of-words models.
Lemmatization	Converts words to their base dictionary form, improving consistency in word representations.	Requires linguistic knowledge and can be computationally expensive.	Applications needing semantic accuracy, such as DL-based text models.
Numbers removal	Reduces dimensionality by eliminating numerical values that might skew analysis.	May remove important numeric information, such as version numbers or error codes.	Sentiment analysis or topic modeling where numbers are not semantically important.
Compound word splitting	Breaks down multi-word expressions into smaller components, enhancing granularity.	Can introduce errors if not context-aware, leading to unnatural splits.	Text mining applications where fine-grained word representations are required.
Stack trace removal	Removes automatically generated error traces, reducing redundancy in bug reports.	May remove useful debugging information that could aid developers.	When focusing on descriptive aspects of bug reports rather than technical traces.
Non-alphanumeric characters removal	Simplifies text representation by stripping special characters.	Can result in loss of structured information, such as code snippets or logs.	When standardizing text for simpler NLP models where symbols do not contribute meaning.

In summary, we obtain the findings shown as below:

Finding 3: Stop word removal remains the most widely used preprocessing method, followed by tokenization and stemming in second and third place, respectively. From 2020 to 2023, the adoption of structural preprocessing methods has seen a significant rise, surpassing their usage in earlier years.

4.2 RQ2: What Software Projects Are Evaluated?

Given the fact that bug reports are commonly used in real-world scenarios, it is important to know in which open source software projects ML-based approaches have been widely applied for automating the analysis task. Thus, in the following, we explore the landscape of bug reporting practices within various software projects.

Overall, we found ML-based bug report analysis has been used in 172 different projects.⁷ Figure 8(a) illustrates the top 10 software projects considered and their number of appearances in the primary studies. As can be seen, Eclipse—a major IDE for Java development—and Mozilla Core are

⁷To ensure fairness, we count open sourced projects only.

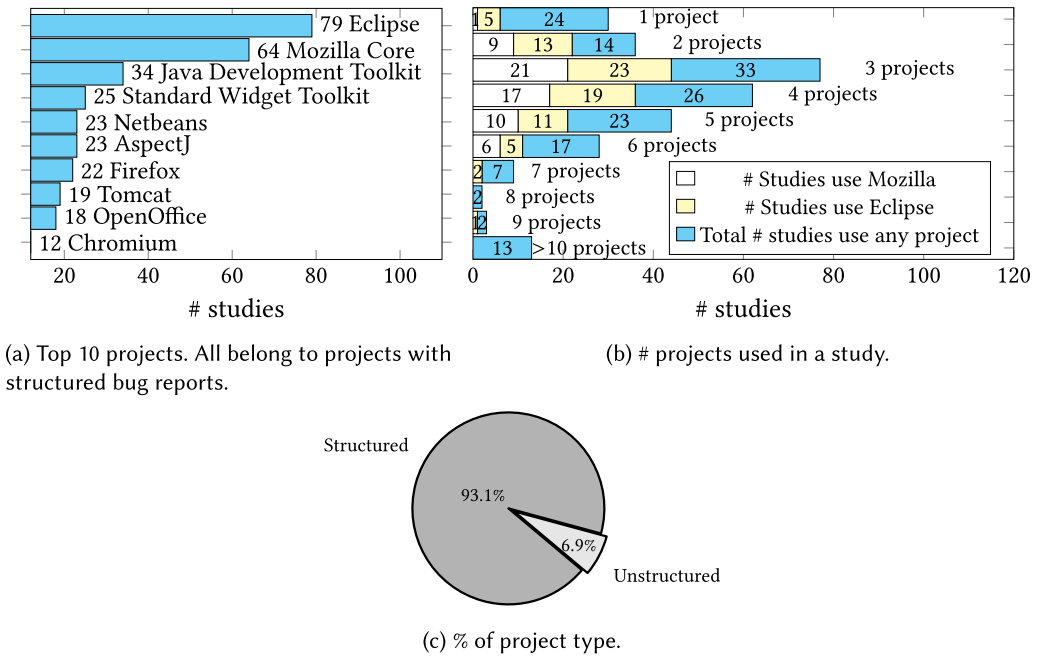


Fig. 8. Statistics of projects used to evaluate the ML algorithms.

the most predominantly used projects in the evaluation, appearing in nearly half of the primary studies. Java Development Toolkit ranks the third, which has significantly outnumbered others in the primary studies. Note that all projects were created before 2014, which is the starting year of this survey.

Throughout our investigation, we encounter two prominent categories of projects: those with structured bug reports and those with unstructured bug reports. The distinction between these categories lies in the methodologies and tools utilized for bug tracking and management. Projects falling into the structured category, such as Eclipse, Mozilla Core, and Java Development Toolkit, often employ platforms like Bugzilla or JIRA, which enforce a structured approach to bug reporting. On the other hand, projects like those hosted on GitHub typically feature unstructured bug reports, characterized by a more flexible and less standardized format. We found that although most projects in primary studies use structured bug reports, a considerable number (13) of primary studies utilized unstructured bug reports during 2020–2023. In contrast, only one paper from 2014–2019 did so.

In this context, we explore the differences and implications of both structured and unstructured bug reporting methodologies. By analyzing a diverse range of software projects, we aim to explain the strengths, weaknesses, and implications of each approach.

4.2.1 Projects with Structured Bug Reports. Structured bug reports help developers and testers communicate more efficiently with predefined bug description items. These reports typically include specific and standardized fields that provide all the information to understand, replicate, and address bugs. The main components of a structured bug report usually contain: title, description, environment, severity, priority, assignee, and so on. These projects predominantly utilize Bugzilla or JIRA to issue tracking and management. Bugzilla stands out as a leading free **Issue Tracking System (ITS)** globally, with major projects of Eclipse, Firefox, Mozilla core, JDT, SWT, AspectJ and Firefox relying on its robust features. A typical bug reporting process in Bugzilla involves comprehensive textual and categorical details, resembling a form-filling activity. Similarly, JIRA has witnessed a surge in adoption over the past decade, with over 65,000 organizations leveraging

its capabilities.⁸ JIRA, akin to Bugzilla, follows a form-filling design for bug report submission, ensuring structured information gathering, which is used by the projects of Netbeans, Tomcat, and OpenOffice. Chromium, however, utilizes a unique platform for its structured ITS, distinct from either Bugzilla or JIRA. This approach allows Chromium to tailor the issue tracking process to meet specific project needs while maintaining the benefits of structured data collection.

4.2.2 Projects with Unstructured Bug Reports. GitHub serves as a prominent Git repository hosting service, offering diverse functionalities including issue tracking. GitHub issues, however, tend to contain unstructured information compared to Bugzilla and JIRA. While textual details are prevalent, categorical fields are often customizable per repository and may not be mandatory or well-defined. Consequently, GitHub lacks the structured nature inherent in Bugzilla and JIRA. Nonetheless, GitHub repositories can utilize labels to categorize issues, albeit in a less systematic manner. The absence of rigid categorical structures in GitHub offers both advantages and disadvantages. On one hand, it simplifies issue reporting by providing flexibility; on the other hand, it complicates the systematic extraction of useful categorical information. Reflecting this trend, Netbeans has also adopted GitHub's ITS since 2022.

Some GitHub repositories, such as the VSCode project, attempt to bridge this gap by offering templates to embed categorical information within textual descriptions. For instance, the VSCode template prompts users to provide details like VSCode version and operating system, enhancing the categorization process. Despite this, adherence to such templates is not obligatory and may be disregarded by issue reporters.

This unique characteristic of GitHub's bug reporting system may facilitate the differentiation between bug reports, feature requests, and documentation updates. However, it also poses challenges in maintaining consistency and extracting structured information systematically.

Discussions. Figure 8(b) shows the number of projects considered in each study. The majority of primary studies utilized datasets comprising three projects, followed by those with four and five projects. In particular, Eclipse and Mozilla are often considered in those cases, and up to 90% of the time they are considered together. In particular, there are 94 studies with Eclipse/Mozilla and they cover 38 projects. For the other 110 without these two, 132 projects are covered. This implies that, current studies tend to believe Eclipse and Mozilla are good representatives of the others, especially when only three projects are used.

Although, as presented in Figure 8(c), most papers over the past decade have adopted projects with structured bug reports. However, the application of projects with unstructured bug reports in the recent 4 years, which indicates an increasing focus on projects employing unstructured bug reports, suggests that more powerful language models should be used in current bug report analysis, as they are better at handling complex unstructured data.

Following a comprehensive review, the advantages, limitations, and best-suited scenarios for each project type have been identified and summarized in Table 5. This table serves as a valuable reference, enabling researchers and practitioners to select the most appropriate projects based on specific situations and requirements.

Overall, we conclude that:

Finding 4: For projects with unstructured bug reports, Eclipse is the predominantly used software project in evaluation. Mozilla Core comes the second. There has been a growing trend in focusing on projects with unstructured bug reports in the past 4 years.

⁸<https://tinyurl.com/y8vhxujv>.

Table 5. Summaries of the Strengths, Weaknesses, and Best-Suited Usage Scenarios for the Projects

Category	Strength	Weakness	Best-Suited Scenario
Structured	Facilitates efficient communication with predefined bug description items, ensuring all necessary information is provided.	May be rigid and less flexible, potentially discouraging detailed reporting.	Projects requiring comprehensive and standardized bug reporting, such as Eclipse and Mozilla Core.
Unstructured	Offers flexibility in issue reporting, allowing for customizable and less standardized formats.	Complicates systematic extraction of useful categorical information and maintaining consistency.	Projects benefiting from flexible reporting, such as those hosted on GitHub, including VSCode.

Table 6. Bug Report Analysis Tasks and Their Popularity in Studies from the Academia

Task	%	Type	Description
Bug categorization	28.2	Discriminative	To distinguish whether a bug report describes a bug or not; or to identify its suitability to a bug type.
Bug localization	21.8	Discriminative	To locate potentially buggy files in a software project given a bug report.
Bug assignment	18.0	Discriminative	To assign bug reports to suitable developers.
Bug severity/priority prediction	14.1	Discriminative	To predict the severity and priority for the bug reports.
Duplicate detection	9.7	Discriminative	To detect bug reports that are duplicates.
Summarization	4.9	Generative	To summarize bug reports in brief.
Others	3.3	Discriminative/Generative	Other bug report tasks.

4.3 RQ3: What Tasks in Bug Report Analysis Are Tackled by ML?

In SE, bug report analysis is a broad topic that involves various tasks. In our RQ3, we seek to understand what tasks in bug report analysis have been specifically investigated and addressed by using ML. Table 6 shows the tasks of bug report analysis tackled by ML, sorted based on their popularity using the number of studies found. As we can see, out of the seven task types, bug categorization tends to be of the highest popularity (28.2% of the primary studies). In general, the bug categorization task may be binary or multi-labeled. The former concerns with identifying whether a given bug reports have indeed contain bugs or not; the latter, in contrast, is about deciding which categories of bug types (e.g., performance bugs, security bugs) that a bug report should be assigned to, and thereby improving the productivity of the developers. The actual categories of bug types used vary widely across different studies and software projects [120]. The popularity of bug categorization tasks may be attributed to their clear problem structure and the availability of labeled datasets, which align well with supervised learning approaches in ML. Bug localization, bug assignment and bug severity/priority prediction account for 21.8%, 18.0%, and 14.1% respectively. In theory, again, all of the above are part of the broad classification task, but they require different sources of features and different types of output form in order to fulfill their purposes. Additionally, a trend has been observed in bug report summarization: from 2014–2019, only one primary study addressed this task, but from 2020–2023, there were nine.

Based on the nature of the tasks and their respective objectives, we have categorized these tasks into two major groups: discriminative bug report analysis and generative bug report analysis.

4.3.1 Discriminative Bug Report Analysis. Discriminative bug report analysis refers to the use of ML techniques to classify, sort, or make decisions about bug reports in SE. This approach

primarily involves identifying distinct characteristics or categories within bug reports and using this information to make informed decisions on handling them. The key tasks typically associated with discriminative bug report analysis include:

Bug categorization [1, 33, 37, 50, 165] is a process in SE where reports submitted by users or developers are analyzed to identify and categorize bugs in software. This process is foundational to maintaining and improving software quality. It requires a thorough understanding and analysis of the structure and content of bug reports to diagnose software issues efficiently. Automatic efficient bug categorization facilitates the rapid identification of software bugs, which, in turn, contributes to enhanced software quality and reliability. By accurately classifying bug reports, development teams can more effectively allocate their resources to address the types of bugs they wish to focus on.

Bug assignment [5, 29, 31, 162, 183] refers to the process of assigning bug reports to appropriate developers or teams responsible for their resolution. Effective bug assignment helps ensure timely bug resolution by directing reports to individuals with the requisite expertise and availability. Automating this task reduces manual effort and accelerates the bug fixing process, thereby improving software quality and developer productivity.

Bug localization [7, 83, 115, 127, 198] involves identifying the specific code components or modules responsible for causing reported software bugs. This task is crucial for developers to isolate and fix defects efficiently. Automated bug localization techniques leverage information from bug reports, source code, and program execution traces to pinpoint the root causes of bugs, facilitating rapid debugging and resolution. Knowing where bugs are likely to be found can help in better allocating testing and debugging resources, focusing efforts where they are most needed.

Bug severity/priority prediction [138, 141, 145, 147, 156] is important for allocating resources effectively and addressing critical issues promptly. Automated prediction models analyze various features of bug reports, such as descriptions, stack traces, and historical data, to assess the severity and prioritize bugs accordingly. By accurately prioritizing bugs, development teams can focus their efforts on resolving critical issues and delivering high-quality software products.

Duplicate bug reports often arise when multiple users encounter the same underlying software defect and submit separate reports. *Duplicate detection* [28, 71, 105, 113, 200] and its management is crucial for avoiding redundancy and optimizing resource utilization in bug resolution efforts. Automated duplicate detection systems compare incoming bug reports with existing reports, leveraging textual similarity and metadata attributes to identify duplicates and streamline the bug triaging process.

These discriminative bug report analysis tasks play a vital role in improving the efficiency, accuracy, and scalability of bug management processes, ultimately enhancing software quality and developer productivity.

4.3.2 Generative Bug Report Analysis. Generative Bug Report Analysis in SE focuses on the creation of new content based on existing bug reports. Unlike discriminative tasks that involve classification and sorting, generative tasks aim to synthesize and produce informative outputs that help in understanding and communicating the essence of bug reports more effectively. This category encompasses tasks such as bug report title generation and bug report summarization, which involves condensing the information contained in bug reports into concise and informative summaries. In this review, we collectively refer to these tasks as *bug report summarization*, recognizing their shared objective of distilling key insights from bug reports.

One notable example of generative bug report analysis is presented in a paper proposing a method for generating human-readable fault reports by leveraging time-series data and textual metadata of metrics for detecting faults in a target system. The authors integrated heterogeneous sources of monitoring data and demonstrated effectiveness through experiments on an e-commerce

service implemented in a Kubernetes cluster [44]. Another noteworthy contribution comes from a study introducing a method named iTAPE, which automatically generates titles for bug reports to aid project practitioners in quickly grasping the core idea of reported bugs. This approach utilizes a Seq2Seq-based model and addresses challenges such as lacking off-the-shelf datasets and handling low-frequency human-named tokens to produce concise and precise bug report titles [25]. Additionally, researchers proposed a deep attention-based summarization model for generating high-quality bug report titles. By employing a robustly optimized BERT approach and a stacked transformer decoder, this model effectively captures contextual semantic information and improves the accuracy of bug report title generation compared to LSTM-based models [111].

Discussion. From the trends over the last 4 years, it is evident that in the realm of generative bug report analysis, recent advancements in NLP, particularly with models like Meta LLaMA 3⁹ and OpenAI GPT-4,¹⁰ offer promising capabilities for automatically generating high-quality bug report summaries. These models leverage sophisticated language understanding and generation techniques to distill the essential information from bug reports into coherent and informative summaries, empowering developers with actionable insights while minimizing manual effort.

While both LLaMA 3 and GPT-4 possess remarkable text generation capabilities, the choice between them may depend on factors such as accessibility, cost, and specific user requirements. LLaMA 3's open source nature provides users with flexibility and affordability, making it a favorable option for experimentation and integration into various software development workflows. On the other hand, GPT-4, despite potentially offering higher accuracy and performance, entails the use of API services, which may incur additional costs but could be justified by its superior capabilities and support. We think generative bug report analysis, facilitated by advanced language models like LLaMA 3 and GPT-4, presents a promising approach to automating bug report summarization and enhancing the efficiency of software development processes. By harnessing the power of NLP, developers can extract actionable insights from bug reports more effectively, leading to improved software quality and developer productivity. Indeed, some of them may belong to the same general category of task from the ML perspective, i.e., classification tasks, such as bug categorization and bug assignment. However, their purposes are different, and thus the features and data labeling involved are also different.

Therefore, we can conclude that:

Finding 5: Bug categorization is the most popular bug analysis task. In contrast, bug report summarization is still niche. However, it has attracted more attention, evidenced by the significantly increasing number from only one primary study 4 years ago to nine recently.

4.4 RQ4: What Types of Bugs Are Analyzed?

Indeed, there are many specific and detailed categorizations of bug types from the literature [120]. In this work, we classify the main bug types addressed in each study based on what has been stated in the papers. Note that when studies specifically stated that certain types of bugs that possess unique characteristics and exert varying impacts on software quality and user experience are focused, we put them into *Specific bug* category. When no bug type has been specifically stated, we put the study into *General bug* category.

⁹<https://llama.meta.com/llama3/>.

¹⁰<https://openai.com/index/gpt-4/>.

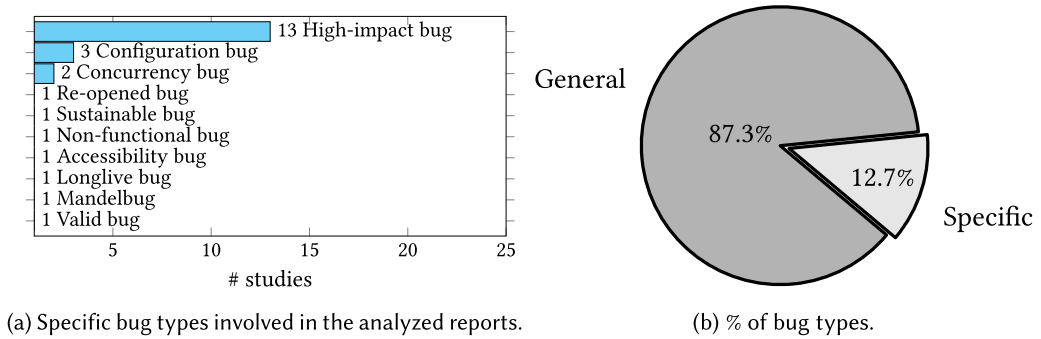


Fig. 9. Statistics of bug types.

In this review, we found that there are 188 studies investigating general bugs with no clear statement on the bug types. Figure 9(a) shows the summary of the results in specific bug types. As we can see, 13 studies have focused on high-impact bug reports. High-impact bugs are composed of an aggregation of various bug types. Our statistical breakdown reveals the presence of one performance bug, two blocking bugs, eight security bugs, one surprise bug, and one breakage bug [81]. Additionally, nine other bug types, including three configuration bugs, two concurrency bugs, and other bug types, each with only one instance, form the minority of non-general bug type studies.

Moreover, the comparative analysis of bug report studies between two distinct periods, 2014–2019 and 2020–2023, reveals notable trends and shifts in the focus of research. During the period from 2014 to 2019, there were 11 primary studies dedicated to specific bug types. In contrast, the period from 2020 to 2023 saw an increase in the number of studies focusing on specific bug types, with 13 primary studies reported.

Next, we will describe these types of bugs in detail:

4.4.1 General Bug Types. Different from specific bug categories, bug report analysis studies encompass a broad spectrum of general bug types. While these types may not be explicitly defined or categorized, they collectively contribute to the understanding of software defects and the improvement of bug management processes. General bug types serve as a baseline for assessing overall software quality, identifying trends, and guiding quality assurance efforts.

4.4.2 Specific Bug Types. Bug reports often encompass a diverse array of issues, each with its unique characteristics and impacts on software quality and user experience. Within the realm of specific bug types, several categories emerge, each addressing distinct aspects of software functionality and behavior. Next, we will introduce each specific bug type we have discovered:

High-impact bugs [81] significantly affect software usability, stability, or security. These include surprise bugs, dormant bugs, blocking bugs, security bugs, performance bugs, and breakage bugs. Analyzing these types is crucial for user satisfaction, system reliability, and organizational reputation.

Configuration bugs arise from incorrect or incompatible software configurations, studied in three works [160, 167, 175]. *Concurrency bugs*, appearing in two studies [27, 197], result from improper handling of concurrent processes, leading to race conditions, deadlocks, or inconsistent states. *Re-opened bugs* refer to issues previously resolved but resurfacing due to incomplete fixes or regressions, studied by Xia et al. [170].

Sustainable bugs persist across multiple versions or releases due to design flaws or complex dependencies. For example, Zhang et al. [193] propose a method to balance accuracy and

sustainability in developer recommendations using DL. *Non-functional bugs* affect attributes like usability, performance, reliability, and maintainability. To address these, Long et al. [107] develop a tool to identify non-functional bug reports in DL frameworks. *Accessibility bugs* hinder access for users with disabilities, requiring solutions for inclusive design. Aljedaani et al. [4] tackle this as a binary classification problem.

Long-lived bugs evade resolution over extended periods. Gomes et al. [50] compare ML algorithms for predicting long-lived bugs. *Mandelbugs* are complex and unpredictable, analyzed using a combined semantic model and DL approach [33]. *Valid bugs* are genuine issues requiring immediate attention. Fan et al. [35] propose a method to automatically verify bug report authenticity.

Discussion. Our study categorizes bugs in ML-assisted bug report analysis into general and specific types, covering a wide range of issues from high-impact bugs to configuration and concurrency bugs. This broad scope demonstrates an evolving awareness of various software quality aspects, including sustainable, non-functional, and accessibility bugs. However, gaps remain, such as the underrepresentation of mandelbugs and long-lived bugs, indicating areas for future research. Identifying common or impactful bug types helps direct analytical efforts and resources more effectively.

Although most papers focus on general bug types as shown in Figure 9(b), the last 4 years have seen a shift towards analyzing specific bug types, reflecting growing interest and sophistication in this area. Researchers are focusing on the nuances of non-functional, performance, and valid bugs, driven by advancements in ML and more available data. Despite this progress, specific bug types still represent a small proportion of research, highlighting an opportunity for further exploration. Expanding ML applications to include more specific bug types could improve bug detection precision and intervention effectiveness, enhancing software quality and user experience. Detailed bug categorization could also foster the development of specialized ML algorithms tailored to diverse issues.

In summary, we conclude that:

Finding 6: Majority of the primary studies have been focusing on general bugs, while 10 other studies work on specific bug types, showcasing an increasing research trend on dealing with specific bug types in recent studies.

4.5 RQ5: Which Evaluation Metrics and Procedures Are Used in Performance Evaluation of Bug Report Analysis?

4.5.1 Evaluation Metrics. It is important to understand what metrics/procedures have been used to evaluate ML-based bug report analysis, as their meaning underpins the conclusions drawn. In this study, we categorize the evaluation metrics used in each paper based on the descriptions provided. If a study explicitly mentions an evaluation metric that is specifically tailored to aspects relevant to bug report analysis processes and strategies, we classify it under *Bug report related evaluation metrics*. Otherwise, we classify the study under *General evaluation metrics*.

General Evaluation Metrics. In this section, we present the top 10 most used evaluation metrics, as shown in Figure 10(a). These metrics are crucial for assessing the performance and effectiveness of bug report analysis tasks. In particular, Precision, F1-score, Accuracy and Recall are equally the most predominated metrics, which are widely used in most primary studies. *Precision* measures the accuracy of positive predictions, used in 107 studies [56, 102, 111, 122, 133]. It is the ratio of true positives to total positive predictions, helping to avoid false positives in bug report analysis.

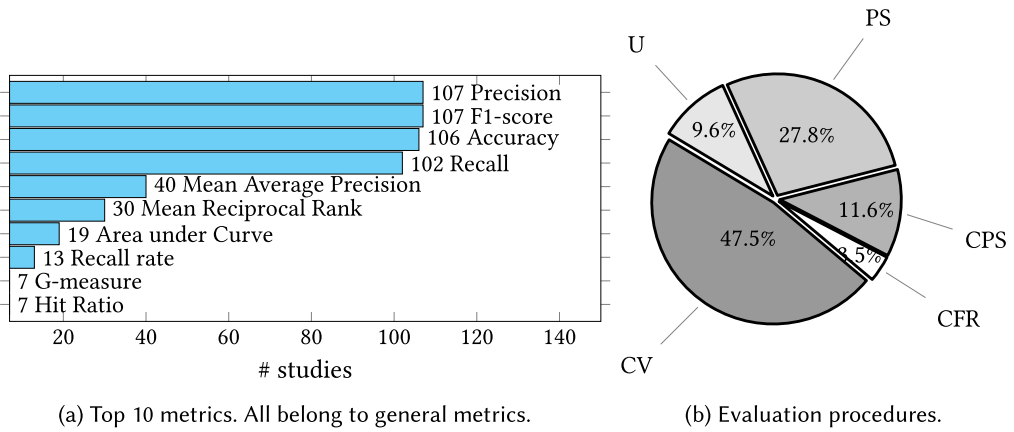


Fig. 10. Statistics of metrics and evaluation procedures.

However, it neglects false negatives. The *F1-score*, the harmonic mean of precision and recall, is used in 107 studies [11, 84, 92, 193, 194]. It balances precision and recall, making it crucial for analyzing bug reports where both false positives and negatives are important, though it may not reflect specific operational priorities. *Accuracy* measures the overall correctness of predictions and is used in 106 studies [10, 68, 117, 118, 134]. It is the ratio of correct predictions to total instances but can be misleading in imbalanced datasets. *Recall*, used in 102 studies [119, 141, 164, 196, 198], measures the system's ability to identify all relevant instances by the ratio of true positives to total actual positives. Its drawback is the potential increase in false positives, complicating bug triage.

Some important but less frequently used metrics include: **Mean Average Precision (MAP)** evaluates the average precision at various recall levels, summarizing the precision-recall tradeoff [26, 88, 128, 131, 157]. However, it might mask variations in performance across different bug report types. **Mean Reciprocal Rank (MRR)** calculates the average reciprocal rank of results for a sample of queries, focusing on the position of the first relevant result [26, 40, 108, 128, 192]. It may overlook the quality of all returned results. **Area under the Curve (AUC)** measures system performance across different thresholds by calculating the area under the ROC curve [34, 48, 50, 130, 140]. However, it focuses on ranking rather than actual predicted probabilities, which may not fully reflect the model's calibration.

Recall rate measures the proportion of true positives out of all actual positives, used in 13 studies [3, 28, 74, 131, 189]. It ensures all relevant bug reports are identified but may lead to many false positives. The *G-measure*, the geometric mean of precision and recall, balances both metrics, giving equal weight to false positives and false negatives. It is used in seven studies [110, 119, 124, 142, 165, 194], but may not align with specific operational priorities. The *Hit Ratio* measures the ratio of successful hits (retrieving at least one relevant bug report) to total queries, used in seven studies [40, 58, 67, 118, 157]. It is useful for finding at least one relevant bug report but overlooks the completeness and relevance of all results.

Bug Report Related Evaluation Metrics. Bug report related evaluation metrics focus specifically on aspects relevant to bug report analysis processes and strategies. These metrics provide insights into the efficiency and effectiveness of bug assignment and prioritization techniques, which are essential for maintaining software quality and reliability.

Overdue Rate measures the percentage of bug reports that were not assigned to developers within the specified timeframe. This metric is essential for assessing the efficiency of bug assignment

strategies [68]. A high overdue rate indicates delays in bug triaging and assignment, potentially leading to prolonged resolution times and increased software instability. By monitoring and reducing the overdue rate, teams can ensure timely allocation of bug reports to developers, thereby enhancing the overall responsiveness of bug handling processes. **Average Percentage of Fault Detected (APFD)** calculates the average percentage of faults detected by a bug prioritization strategy. This metric considers the order in which faults are detected during the inspection process, reflecting the effectiveness of prioritization techniques in identifying critical bugs early [147]. A higher APFD score indicates that the prioritization strategy successfully detected faults sooner in the inspection process, leading to faster bug resolution and improved software quality. By optimizing bug prioritization strategies to maximize APFD scores, teams can prioritize resources effectively and focus on resolving the most impactful bugs promptly.

Discussion. In fact, the result that F1-score, Recall, Precision, and Accuracy are nearly equally the most predominated metrics is not surprising, as it is known that these metrics are effective for classification tasks in ML [41]. Accuracy, which measures the ratio of correct predictions over the total number of instances evaluated, is used quite often in many cases, but still significantly less common than the top three. This is not surprising either, as it has been shown that accuracy has less discriminating power than F1-score in most cases [78]. Meanwhile, despite their importance, bug report related evaluation metrics have not received significant attention in the bug report analysis literature. We found only two articles that utilized these unique metrics, indicating a lack of emphasis on evaluating bug assignment and prioritization strategies. Researchers and practitioners in this field should explore and employ these metrics more frequently to improve bug handling processes and enhance software quality.

The comprehensive review has revealed the unique strengths, weaknesses, and best-fit applications for each evaluation metric. These insights are succinctly captured in Table 7, serving as a practical guide for researchers and practitioners to determine the most suitable metric for evaluation in various contexts.

4.5.2 Evaluation Procedures. A related topic to the metrics is the evaluation procedures, which describe how the results measured by the metrics are validated using different data samples. This is non-trivial for providing reliable evaluation, given the stochastic nature of most ML algorithms.

In Figure 10(b), we summarize the four different categories of evaluation procedures adopted in the primary studies, each of them are briefly explained as below:

- *k-Fold CV*: The training data is separated into k -folds. Each single fold is used for testing the model once, and the remaining $k - 1$ folds are used as training data. Overall results of all folds are reported.
- **Chronologically Fold Rotation (CFR)**: All training data is divided into k equally sized folds in chronological order, and three types are used in primary studies:
 - The first i folds form the training set and the $fold_{i+1}$ is used as a test set [64, 186, 188, 191].
 - Train a model on $fold_i$ and test it on $fold_{i+1}$ [146].
 - Trained a model from $fold_i$ to $fold_{i+j-1}$ and test it on $fold_{i+j}$, for all $1 \leq i \leq k - j$ [146].
 The overall results from the runs are reported.
- **Chronologically Percent Split (CPS)**: Similar to the above, but instead of using folds, different percentages of split are used.
- **Percent Split (PS)**: The training data is randomly split for training and testing to get the overall result, similar to multiple repeated hold-out.
- **Unspecified (U)**: The evaluation procedure has not been clearly stated.

Table 7. Summaries of the Strengths, Weaknesses, and Best-Suited Usage Scenarios for the Top 10 Metrics

Category	Strength	Weakness	Best-Suited Scenario
Precision	Measures the accuracy of positive predictions, helping to avoid false positives.	Neglects false negatives.	Scenarios where avoiding false positives is crucial, such as bug report categorization.
F1-score	Balances precision and recall, making it crucial for analyzing both false positives and negatives.	May not reflect specific operational priorities.	Scenarios where both false positives and negatives are important, such as bug severity/priority prediction.
Accuracy	Measures the overall correctness of predictions.	Can be misleading in imbalanced datasets.	General performance assessment in balanced datasets.
Recall	Measures the system's ability to identify all relevant instances.	Potential increase in false positives.	Scenarios where identifying all relevant instances is important, such as bug assignment.
MAP	Evaluates the average precision at various recall levels, summarizing the precision-recall tradeoff.	Might mask variations in performance across different bug report types.	Scenarios requiring a summary of precision-recall tradeoffs, such as bug report classification.
MRR	Calculates the average reciprocal rank of results, focusing on the position of the first relevant result.	May overlook the quality of all returned results.	Scenarios where the position of the first relevant result is important, such as duplicate detection.
AUC	Measures system performance across different thresholds by calculating the area under the ROC curve.	Focuses on ranking rather than actual predicted probabilities.	Scenarios requiring performance assessment across different thresholds, such as bug report classification.
Recall rate	Measures the proportion of true positives out of all actual positives.	May lead to many false positives.	Scenarios where ensuring all relevant bug reports are identified, such as duplicate detection.
G-measure	Balances precision and recall, giving equal weight to false positives and false negatives.	May not align with specific operational priorities.	Scenarios requiring a balanced assessment of precision and recall, such as bug report classification.
Hit ratio	Measures the ratio of successful hits (retrieving at least one relevant bug report) to total queries.	Overlooks the completeness and relevance of all results.	Scenarios where finding at least one relevant bug report is important, such as initial bug assignment.

Discussion. Clearly, we note that CV (normally 10 folds) [41] is the evaluation procedure used in the majority of the primary studies (47.5%), together with the evaluation metrics, particularly the Recall, Precision, and F1-score. The other procedures form the minority with small margins between each other, including seven studies whose evaluation procedures have not been clearly stated.

However, with the increasing application of DL models, CV may become time-consuming. DL models typically require large datasets to achieve optimal performance, and implementing CV on such large datasets significantly increases computational pressure. Each fold in the CV process necessitates retraining the model from scratch, which is computationally intensive and time-consuming, particularly for complex DL architectures. This constraint can limit the feasibility of using CV as a standard evaluation method for DL models, prompting the need for alternative strategies that balance thorough model evaluation with computational efficiency.

Drawing from the review results, Table 8 encapsulates the strengths, weaknesses, and ideal scenarios for each evaluation procedure. This table acts as a valuable resource, enabling researchers and practitioners to identify the most appropriate evaluation method for various contexts and requirements.

Table 8. Summaries of the Strengths, Weaknesses, and Best-Suited Usage Scenarios for the Evaluation Procedures

Category	Strength	Weakness	Best-Suited Scenario
CV	Provides a comprehensive evaluation by using all data for both training and testing across different folds.	Can be computationally intensive, especially with large datasets.	General performance assessment where reliable evaluation is crucial.
PS	Simple and quick to implement, providing a straightforward evaluation.	May not provide a reliable evaluation due to random splits and potential data imbalance.	Initial model evaluation and scenarios with limited computational resources.
CPS	Maintains the temporal order of data, providing a realistic evaluation for time-dependent data.	May not be suitable for non-temporal data and can be less flexible.	Evaluating models on time-series data or scenarios where chronological order is important.
U	Flexibility in evaluation procedures, allowing for customized approaches.	Lack of standardization can lead to inconsistent and unreliable results.	Scenarios where specific evaluation procedures are not critical or well-defined.
CFR	Provides a realistic evaluation by maintaining the chronological order of data and using different fold rotations.	Can be complex to implement and computationally intensive.	Evaluating models on time-series data or scenarios requiring realistic temporal evaluation.

Bringing together the above result from Sections 4.5.1 and 4.5.2, we can conclude that:

Finding 7: Precision, F1-score, accuracy, and recall are key metrics for evaluating ML models in bug report analysis tasks. Bug report related evaluation metrics have not received significant attention in the bug report analysis literature. The majority of studies prefer k -fold CV for model evaluation.

4.6 RQ6: Which Statistical Tests and Standardized Effect Size Measurements Are Applied to Mitigate Bias?

Statistical tests and standardized effect size measurements are essential to ensure the reliability of results, especially given that the stochastic nature of ML training pipelines. In Figure 11(a) and (b), we summarize the statistical tests and effect size measurement methods used in the primary studies. As can be seen, although the results have shown that Wilcoxon signed-rank test is more commonly used than the others, surprisingly, a significant number of studies (128) have completely ignored the importance of statistical tests. As for the standardized effect size, most primary studies have not even mentioned such, constituting more than 90% of the studies.

First, we examine the statistical tests employed in bug report analysis, which serve as crucial tools for evaluating the significance of findings and drawing reliable conclusions. These tests are categorized based on their applicability and assumptions. The following statistical tests are commonly utilized in bug report analysis:

4.6.1 Statistical Test. The *Wilcoxon Signed-Rank Test* is a non-parametric test comparing two related samples or repeated measurements from the same group to assess if their medians differ significantly. This test is useful for evaluating performance differences between two algorithms or models on the same dataset, cited in 30 studies [48, 50, 92, 115, 128]. Its main disadvantage is the requirement for symmetrical data distribution around the median. The *Mann-Whitney U Test* compares two independent samples non-parametrically, appearing in 19 studies [34, 40, 66, 86, 122].

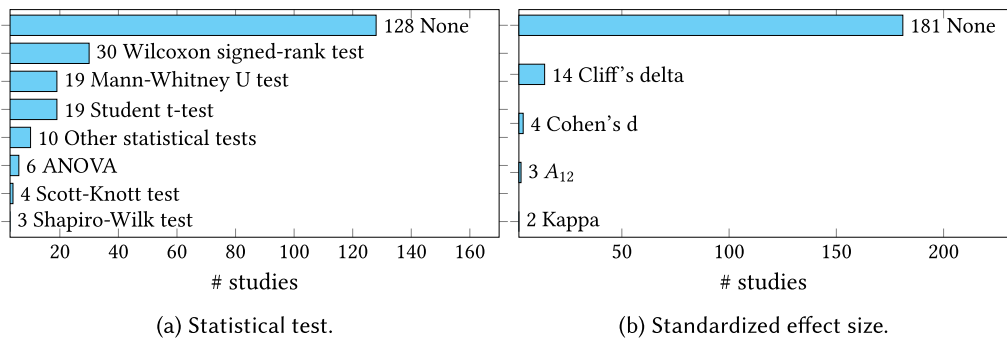


Fig. 11. Statistics of statistical test and standardized effect size.

It evaluates if the distributions of two groups are the same or if one group tends to have higher values. A limitation is its sensitivity to sample size differences. The *Student t-Test* determines if there's a significant difference between the means of two independent groups, used in 19 studies [63, 119, 160, 176, 186]. It assumes normal data distribution, requiring a normality test before application, which can limit its use. *ANOVA* analyzes differences among group means in a sample, cited in six studies [2, 126, 150, 151, 155, 184]. It compares three or more groups simultaneously, but requires variance homogeneity among groups. The *Scott-Knott Test*, used post-ANOVA, identifies significant differences between group means by partitioning them into subgroups, used in four studies [9, 107, 147, 196]. It handles data heterogeneity well but is sensitive to outliers. The *Shapiro-Wilk Test* assesses the normality of data samples, adopted in three studies [176, 177, 179]. It validates assumptions for parametric tests like the *t*-test and ANOVA, but its sensitivity can lead to incorrect normality rejections with larger samples.

4.6.2 Standard Effect Size Measurements. Furthermore, we explain various effect size measurements commonly employed in bug report analysis. These measurements are pivotal for assessing the significance and magnitude of differences observed in bug-related metrics, aiding in the interpretation and comparison of results across different studies. We categorize these measurements into several widely used techniques:

Cliff's delta is a non-parametric measure used in 14 studies [104, 161, 169, 178, 197] to quantify effect size by comparing the probabilities of observations between groups. It is robust against non-normal distributions, making it suitable for diverse bug report analyses. However, it may be less informative with very small sample sizes. *Cohen's d*, used in four studies [111, 140, 147, 150], measures effect size as the difference between two means in standard deviation units. It helps gauge practical significance but assumes homogeneous, normally distributed data. A_{12} quantifies the strength of an effect, used in three studies, and helps evaluate the effectiveness of bug report analysis strategies but does not indicate effect direction. *Kappa*, found in two studies [2, 61], measures inter-rater agreement for categorical items, adjusting for chance agreement. It is useful for assessing consistency in bug report analysis but can be inaccurate with high prevalence or extreme distributions.

Discussion. Our analysis reveals a varied landscape in the utilization of statistical tests and effect size measurements within the field of bug report analysis. Notably, while a range of statistical tests such as the Wilcoxon Signed-Rank Test, Mann-Whitney U Test, Student *t*-Test, ANOVA, Scott-Knott Test, and Shapiro-Wilk Test have been applied in numerous studies, a substantial proportion (59.8%) of the research does not employ these robust methodologies. This omission may

Table 9. Summaries of the Strengths, Weaknesses, and Best-Suited Usage Scenarios for the Most Common Statistical Tests

Category	Strength	Weakness	Best-Suited Scenario
Wilcoxon signed-rank test	Non-parametric test comparing two related samples or repeated measurements to assess median differences.	Requires symmetrical data distribution around the median.	Evaluating performance differences between two algorithms or models on the same dataset.
Mann-Whitney U test	Non-parametric test comparing two independent samples to evaluate if their distributions are the same.	Sensitive to sample size differences.	Comparing distributions of two independent groups.
Student <i>t</i> -test	Determines if there's a significant difference between the means of two independent groups.	Assumes normal data distribution, requiring a normality test before application.	Assessing mean differences between two groups with normally distributed data.
ANOVA	Analyzes differences among group means in a sample, comparing three or more groups simultaneously.	Requires variance homogeneity among groups.	Comparing means across multiple groups.
Scott-Knott test	Identifies significant differences between group means by partitioning them into subgroups.	Sensitive to outliers.	Post-ANOVA analysis to identify significant group differences.
Shapiro-Wilk test	Assesses the normality of data samples, validating assumptions for parametric tests.	Sensitivity can lead to incorrect normality rejections with larger samples.	Validating normality assumptions for parametric tests like the <i>t</i> -test and ANOVA.

undermine the reliability and generalizability of the findings, especially given the stochastic nature of the underlying algorithms in ML applications.

Similarly, standardized effect size measurements of Cliff’s delta, Cohen’s *d*, A_{12} , and Kappa, though utilized in several studies, are conspicuously absent in the majority. This lack of standardized effect size reporting limits the ability of the research community to accurately assess the magnitude and practical significance of the findings, an essential aspect of empirical research.

The general neglect of statistical tests and effect size measurements suggests a need for more rigorous methodological standards in the domain of bug report analysis. Encouraging the adoption of a broader array of statistical tests and effect size measurements could enhance the interpretability and comparability of results across studies, fostering a more robust scientific dialogue and advancing the field.

The comprehensive review has highlighted the unique benefits, drawbacks, and ideal applications for various model optimization methods and activation functions. These insights are clearly presented in Tables 9 and 10, offering a practical guide for researchers and practitioners to choose the most suitable statistical test and effect size test in different contexts.

In summary, we therefore have:

Finding 8: Wilcoxon signed-rank test and Cliff’s delta are the most commonly used statistical test methods and effect size measurements. A significant proportion of studies have completely ignored the importance of statistical tests. Effect size has not been reported for most of the work.

5 Future Research Opportunities

5.1 Transformer-Based Model Application for Bug Report Analysis

As discussed in Section 4.1, the landscape of ML in bug report analysis has witnessed significant advancements, particularly with the adoption of Transformer-based models, which offer promising

Table 10. Summaries of the Strengths, Weaknesses, and Best-Suited Usage Scenarios for the Standardized Effect Size Tests

Category	Strength	Weakness	Best-Suited Scenario
Cliff's delta	Robust against non-normal distributions, suitable for diverse bug report analyses.	May be less informative with very small sample sizes.	Quantifying effect size in non-normal distributions.
Cohen's d	Measures effect size as the difference between two means in standard deviation units.	Assumes homogeneous, normally distributed data.	Gauging practical significance in normally distributed data.
A_{12}	Quantifies the strength of an effect, helping evaluate the effectiveness of strategies.	Does not indicate effect direction.	Evaluating the effectiveness of bug report analysis strategies.
Kappa	Measures inter-rater agreement for categorical items, adjusting for chance agreement.	Can be inaccurate with high prevalence or extreme distributions.	Assessing consistency in bug report analysis.

enhancements over traditional methods like CNNs, LSTMs, and SVMs. These models excel in handling long-range dependencies and large datasets, which are common in bug report analysis. Despite existing research in areas such as bug report generation [44], security bug report prediction [194], and bug severity prediction [159], many tasks remain unexplored. Future research should leverage the capabilities of Transformer architectures like BERT and GPT to tackle complex tasks such as bug triaging, duplicate report detection, and summarization of bug reports. Further exploration into the application of Transformers could significantly improve the accuracy and efficiency of parsing and understanding the textual context data within bug reports. Innovations could include the development of domain-specific models that are pre-trained on software development texts to better grasp the context and jargon specific to certain software projects. By effectively harnessing these models, researchers and practitioners could achieve greater precision in bug report analysis tasks, leading to faster and more efficient bug resolution processes. This would not only enhance the productivity of developers but also improve the overall software quality, making this an exciting avenue for future research.

5.2 Automatic Bug Report Analysis Tools Deployed in GitHub

As analyzed in Section 4.2, our survey reveals a growing trend in the use of unstructured bug reports from platforms like GitHub. While structured bug reports benefit from predefined fields and standardized formats, unstructured reports are inherently more variable and complex, posing unique challenges for automated analysis.

Despite the above, we found only one study that design solution explicitly for structured bug reports: Xiao et al. [174] propose CNN Forest, which combines a CNN with an ensemble of random forests, achieving strong performance in semantic parsing and structural information extraction to enhance bug report localization.

Indeed, tools developed for analyzing structured bug reports can also be adapted to handle unstructured reports. For example, Zhang et al. [189] not only applied state-of-the-art duplicate bug report detection methods to structured bug reports from Bugzilla and Jira, but also to unstructured bug reports from GitHub. However, as Zhang et al. [189] observed, unstructured bug reports lack well-defined fields for maintaining duplicate references. As a result, it is difficult to extract duplicate issues following GitHub's guidelines for marking duplicates. This implies that applying tools designed for structured reports to unstructured data introduces two key limitations:

- *Inconsistent Data Formatting*: Structured tools rely on standardized fields, which are absent in unstructured reports. This inconsistency can lead to difficulties in accurately parsing and analyzing the data. For example, structured bug reports (e.g., from Jira) have standardized fields such as steps, expected outcomes, and environment, enabling accurate parsing. In contrast, unstructured reports (e.g., from GitHub) use free-form text without standardization without these fields, complicating data analysis and interpretation.
- *Incomplete Information Extraction*: Unstructured reports may lack critical details or present information in a non-standard manner, making it challenging for structured tools to extract necessary data effectively. For example, structured reports (e.g., from Bugzilla) require reporters to specify critical details such as OS version, severity, and reproduction steps explicitly, enabling efficient data extraction. Conversely, unstructured reports (e.g., from GitHub issues) often omit or vaguely describe essential details, hindering automated tools from effectively extracting key information.

To overcome these limitations, future research should focus on developing advanced analysis tools tailored specifically for unstructured bug reports. Leveraging state-of-the-art NLP techniques, such as Transformer-based architectures and LLMs (e.g., BERT, GPT-4), can enhance the parsing and classification of the nuanced content found in GitHub issues. Additionally, conducting in-depth case studies and application-driven evaluations of such tools will provide concrete insights into their efficacy and practicality in real-world settings. Exploring hybrid approaches that combine rule-based and learning-based methods may offer balanced solutions to handle the variability inherent in unstructured bug reports.

5.3 Bug Report Analysis with LLMs

As emphasized in Section 4.3, the incorporation of LLMs such as GPT-4 or LLAMA 3 into bug report analysis presents a cutting-edge frontier. These models' deep understanding of natural language makes them ideally suited for bug report analysis tasks. However, no primary studies have explored this area, indicating that this field has not kept up with recent trends. To bridge this gap, future research could explore the automation of these discriminative tasks using LLMs to enhance accuracy and process efficiency.

Moreover, LLMs could be pivotal in generative tasks within bug report analysis, such as summarizing complex reports or generating descriptive titles. Such applications could significantly reduce the workload on developers by automating routine analyses and allowing them to focus on more critical tasks. Exploring the use of LLMs to link bug reports with specific code commits or development tasks could also provide groundbreaking improvements in how bugs are traced and resolved.

Meanwhile, predictive tasks that estimate bug severity or prioritize issues based on historical data could benefit from the advanced capabilities of LLMs. LLMs can easily be used for the fields in bug reports, aiding in automated classification efficiently. The potential for LLMs to transform bug report analysis is vast, promising substantial advances in software maintenance and development efficiency.

5.4 Bug Report Analysis for Specific Bugs

As identified in Section 4.4, only 10% of primary studies have focused on analyzing specific types of bugs. This observation underscores a substantial opportunity for researchers to explore the investigation of various specific bug types more thoroughly.

For example, future research could prioritize the development of sophisticated ML algorithms that can identify and classify a wider range of specific bug types. By doing so, it will be possible to address less common but potentially critical specific bugs that may be overlooked by more generalized

models. This focused approach could also facilitate the creation of specialized diagnostic tools that provide developers with precise insights into the nature of the bugs, enabling more targeted and effective interventions.

Moreover, there is significant potential in employing advanced analytics to understand the underlying patterns and causes of specific bugs. Researchers could utilize data mining techniques to extract valuable insights from historical specific bug data, helping to predict the likelihood of specific bug occurrences and their potential impacts on software systems. Such strategic research endeavors will not only fill the existing gap but will also elevate the standards and capabilities of specific bug report analysis frameworks.

5.5 Dedicated Metrics for Bug Report Analysis

As noted in Section 4.5, aside from a few studies using dedicated metrics for bug report analysis like Overdue Rate [68] and APFD [147], there is a lack of research utilizing bug report-related metrics. Creating a dedicated metric for evaluating the performance of bug report analysis tools could lead to more detailed insights and improvements in software quality assurance. This type of metric would ideally reflect the unique aspects of bug reports, such as their urgency, complexity, and impact on the software project.

Hence, we believe that future research should focus on developing composite metrics that integrate various dimensions of bug report analysis, such as the accuracy of bug report generation, the similarity of duplicate bug reports, and the relevance of the recommendations provided. This would provide a more holistic view of a tool's performance and its practical value to developers. A possible opportunity is to consider multiple metrics simultaneously and their joint assessment [98].

Additionally, it would be beneficial to validate these metrics through extensive empirical studies, involving diverse software projects and development environments. This would ensure that the metrics are robust and universally applicable across different contexts and technologies.

5.6 Explainable DL for Bug Report Analysis

Integrating explainability into DL models used in bug report analysis is crucial for their acceptance and effectiveness in practical settings. Researchers should focus on developing methods that not only perform well but also provide transparent and understandable decision-making processes.

This could involve using techniques such as attention mechanisms that highlight the parts of the bug report most relevant to the model's decision, or developing hybrid models that combine DL with rule-based reasoning to provide clear explanations for their outputs.

Moreover, collaborations between ML researchers, software developers, and end-users are essential to ensure that the explanations provided by these models are meaningful and actionable in real-world software development scenarios. This interdisciplinary approach can help refine the models to better meet the practical needs of software maintenance and development.

6 Threats to Validity

Threats to construct validity can be raised by the research methodology applied. We have mitigated such threats by following the systematic review protocol proposed by Kitchenham et al. [89].

Threats to internal validity may be introduced by having inappropriate classification and interpretation of the studies. We have limited this by conducting three iterations of paper reviews by the authors. Error checks and investigations were also conducted to correct any issues found during the search.

Threats to external validity may restrict the generalizability of the revealed statistics and the justification of future research opportunities. We have mitigated such by conducting the survey wider

and deeper: it covers 1,825 searched papers published between 2014 and 2023 from three repositories; while at the same time, extracting 204 prominent primary studies following the exclusion and inclusion procedure. This, together with the evidence from the best practice in SE and ML, have provided rich sources for us to discuss the results and the future research opportunities identified.

7 Conclusion

In this work, we perform a SLR on bug report analysis with ML over the past 10 years. The survey covers 1,825 papers from three repositories, based on which we extracted a more precise set of 585 candidate studies, and finally, 204 primary studies were selected for detailed analysis. Notably, we obtain the following key findings for our RQs:

- *RQ1*: CNN, LSTM and k NN are the top three commonly used ML algorithms for bug report analysis. Word2vec and TF-IDF are the most widely used methods to represent features. Stop word removal is the most widely used preprocessing method, followed by tokenization and stemming.
- *RQ2*: Eclipse and Mozilla Core are the most widely evaluated software projects.
- *RQ3*: Bug categorization task receives the most attention.
- *RQ4*: Majority of the studies focuses on general bug types without a specific focus.
- *RQ5*: Precision, F1-score, accuracy, and recall are much more popular than the other metrics in evaluation. The majority of studies prefer k -fold CV for model evaluation.
- *RQ6*: Statistical test and effect size are ignored by majority of the studies.

Based on the result analysis of the primary studies, we offer insights for each RQ. Additionally, we propose six future research directions. Through the statistics and future research directions discussed in this work, we hope to provide insights and visionary impact that excite a more significant growth of research on specialized ML for bug report analysis, as well as its interaction to the other SE problems, such as configuration management [16, 20, 21, 22, 53, 103, 172], general software optimization [19], performance prediction [51, 52, 54], testing [32, 109], requirements handling [18], and self-adaptation [14, 15, 23, 24, 181].

References

- [1] Petar Afric, Davor Vukadin, Marin Silic, and Goran Delac. 2023. Empirical study: How issue classification influences software defect prediction. *IEEE Access* 11 (2023), 11732–11748.
- [2] Karan Aggarwal, Tanner Rutgers, Finbarr Timbers, Abram Hindle, Russell Greiner, and Eleni Stroulia. 2015. Detecting duplicate bug reports with software engineering domain knowledge. In *Proceedings of the 2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER '15)*, 211–220.
- [3] Thangarajah Akilan, Dhruvit Shah, Nishi Patel, and Rinkal Mehta. 2020. Fast detection of duplicate bug reports using LDA-based topic modeling and classification. In *Proceedings of the 2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*.
- [4] Wajdi Aljedaani, Mohamed Wiem Mkaouer, Stephanie Ludi, Ali Ouni, and Ilyes Jenhani. 2022. On the identification of accessibility bug reports in open source systems. In *Proceedings of the 19th Web for All Conference (W4A '22)*.
- [5] Bader Alkhazi, Andrew DiStasi, Wajdi Aljedaani, Hussein Alrubaye, Xin Ye, and Mohamed Wiem Mkaouer. 2020. Learning to rank developers for bug report assignment. *Applied Software Computing* 95 (2020), 106667.
- [6] Bui Thi Mai Anh and Nguyen Viet Luyen. 2021. An imbalanced deep learning model for bug localization. In *Proceedings of the 28th Asia-Pacific Software Engineering Conference Workshops (APSEC Workshops '21)*.
- [7] Lorena Arcega, Jaime Font, and Carlos Cetina. 2018. Evolutionary algorithm for bug localization in the reconfigurations of models at runtime. In *Proceedings of the 21st ACM/IEEE International Conference on Model Driven Engineering Languages and Systems of the (MODELS '18)*.
- [8] Pasquale Ardimento and Andrea Dinapoli. 2017. Knowledge extraction from on-line open source bug tracking systems to predict bug-fixing time. In *Proceedings of the 7th International Conference on Web Intelligence, Mining and Semantics*, 1–9.

- [9] Maram Assi, Safwat Hassan, Stefanos Georgiou, and Ying Zou. 2023. Predicting the change impact of resolving defects by leveraging the topics of issue reports in open source software systems. *ACM Transactions on Software Engineering and Methodology* 32, Article 14 (2023), 1–34.
- [10] Thazin Win Win Aung, Yao Wan, Huan Huo, and Yulei Sui. 2022. Multi-triage: A multi-task learning framework for bug triage. *Journal of Systems and Software* 184 (2022), 111133.
- [11] Aladdin Baarah, Ahmad Aloqaily, Hala Zyod, and Nasser Mustafa. 2021. Sentiment-based neural network approach for predicting the severity of bug reports. In *Proceedings of the 2021 5th International Conference on Intelligent Computing in Data Sciences (ICDS)*.
- [12] Sean Banerjee, Zahid Syed, Jordan Helmick, Mark Culp, Kenneth Ryan, and Bojan Cukic. 2017. Automated triaging of very large bug repositories. *Information and Software Technology* 89 (2017), 1–13.
- [13] Oscar Chaparro, Jing Lu, Fiorella Zampetti, Laura Moreno, Massimiliano Di Penta, Andrian Marcus, Gabriele Bavota, and Vincent Ng. 2017. Detecting missing information in bug descriptions. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 396–407.
- [14] Tao Chen and Rami Bahsoon. 2017. Self-adaptive and online QoS modeling for cloud-based software services. *IEEE Transactions on Software Engineering* 43, 5 (2017), 453–475. DOI: <https://doi.org/10.1109/TSE.2016.2608826>
- [15] Tao Chen and Rami Bahsoon. 2017. Self-adaptive trade-off decision making for autoscaling cloud-based services. *IEEE Transactions on Services Computing* 10, 4 (2017), 618–632. DOI: <https://doi.org/10.1109/TSC.2015.2499770>
- [16] Pengzhou Chen and Tao Chen. 2026. PromiseTune: Unveiling causally promising and explainable configuration tuning. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM.
- [17] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, 785–794.
- [18] Tao Chen and Miqing Li. 2023. Do performance aspirations matter for guiding software configuration tuning? An empirical investigation under dual performance objectives. *ACM Transactions on Software Engineering and Methodology* 32, 3, Article 68 (2023), 1–41. DOI: <https://doi.org/10.1145/3571853>
- [19] Tao Chen and Miqing Li. 2023. The weights can be harmful: Pareto search versus weighted search in multi-objective search-based software engineering. *ACM Transactions on Software Engineering and Methodology* 32, 1, Article 5 (2023), 1–40. DOI: <https://doi.org/10.1145/3514233>
- [20] Tao Chen and Miqing Li. 2024. Adapting multi-objectivized software configuration tuning. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 539–561. DOI: <https://doi.org/10.1145/3643751>
- [21] Pengzhou Chen, Tao Chen, and Miqing Li. 2024. MMO: Meta multi-objectivization for software configuration tuning. *IEEE Transactions on Software Engineering* 50, 6 (2024), 1478–1504. DOI: <https://doi.org/10.1109/TSE.2024.3388910>
- [22] Pengzhou Chen, Jingzhi Gong, and Tao Chen. 2025. Accuracy can lie: On the impact of surrogate model in configuration tuning. *IEEE Transactions on Software Engineering* 51, 2 (2025), 548–580. DOI: <https://doi.org/10.1109/TSE.2025.3525955>
- [23] Tao Chen, Ke Li, Rami Bahsoon, and Xin Yao. 2018. FEMOSAA: Feature-guided and knee-driven multi-objective optimization for self-adaptive software. *ACM Transactions on Software Engineering and Methodology* 27, 2, Article 5 (2018), 1–50. DOI: <https://doi.org/10.1145/3204459>
- [24] Tao Chen. 2022. Lifelong dynamic optimization for self-adaptive systems: Fact or fiction? In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '22)*. IEEE, 78–89. DOI: <https://doi.org/10.1109/SANER53432.2022.00022>
- [25] Songqiang Chen, Xiaoyuan Xie, Bangguo Yin, Yuanxiang Ji, Lin Chen, and Baowen Xu. 2020. Stay professional and efficient: Automatically generate titles for your bug reports. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20)*.
- [26] Hao Chen, Haiyang Yang, Zilun Yan, Li Kuang, and Lingyan Zhang. 2022. CGMBL: Combining GAN and method name for bug localization. In *Proceedings of the 22nd IEEE International Conference on Software Quality, Reliability and Security (QRS '22)*, 231–241.
- [27] Paolo Ciancarini, Francesco Poggi, Davide Rossi, and Alberto Sillitti. 2017. Analyzing and predicting concurrency bugs in open source systems. In *Proceedings of the 2017 International Joint Conference on Neural Networks (IJCNN '17)*, 721–728.
- [28] Nathan Cooper, Carlos Bernal-Cárdenas, Oscar Chaparro, Kevin Moran, and Denys Poshyvanyk. 2021. It takes two to TANGO: Combining visual and textual information for detecting duplicate video-based bug reports. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE '21)*.
- [29] Jie Dai, Qingshan Li, Hui Xue, Zhao Luo, Yinglin Wang, and Siyuan Zhan. 2023. Graph collaborative filtering-based bug triaging. *Journal of Systems and Software* 200 (2023), 111667.
- [30] Jayati Deshmukh, Sanjay Podder, Shubhashis Sengupta, and Neville Dubash. 2017. Towards accurate duplicate bug retrieval using deep learning techniques. In *Proceedings of the 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 115–124.

- [31] Atish Kumar Dipongkor, Md. Saiful Islam, Ishtiaque Hussain, Sira Yongchareon, and Sajib Mistry. 2023. On fusing artificial and convolutional neural network features for automatic bug assignments. *IEEE Access* 11 (2023), 49493–49508.
- [32] Chengwen Du and Tao Chen. 2024. Contexts matter: An empirical study on contextual influence in fairness testing for deep learning systems. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '24)*. Xavier Franch, Maya Daneva, Silverio Martínez-Fernández, and Luigi Quaranta (Eds.), ACM, 107–118. DOI: <https://doi.org/10.1145/3674805.3686673>
- [33] Xiaoting Du, Zheng Zheng, Guanping Xiao, Zenghui Zhou, and Kishor S. Trivedi. 2022. DeepSIM: Deep semantic information-based automatic mandelbug classification. *IEEE Transactions on Reliability* 71 (2022), 1540–1554.
- [34] Amir Elmishali and Meir Kalech. 2023. Issues-driven features for software fault prediction. *Information and Software Technology* 155 (2023), 107102.
- [35] Yuanrui Fan, Xin Xia, David Lo, and Ahmed E. Hassan. 2020. Chaff from the wheat: Characterizing and determining valid bug reports. *IEEE Transactions on Software Engineering* 46 (2020), 495–525.
- [36] Qiang Fan, Yue Yu, Gang Yin, Tao Wang, and Huaimin Wang. 2017. Where is the road for issue reports classification based on text mining? In *Proceedings of the 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '17)*.
- [37] Sen Fang, Tao Zhang, Youshuai Tan, He Jiang, Xin Xia, and Xiaobing Sun. 2023. RepresentThemAll: A universal learning representation of bug reports. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE '23)*.
- [38] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020 (Findings of ACL, EMNLP '20)*. Trevor Cohn, Yulan He, and Yang Liu (Eds.), Association for Computational Linguistics, 1536–1547.
- [39] Adrian-Catalin Florea, John Anvik, and Razvan Andonie. 2017. Spark-based cluster implementation of a bug report assignment recommender system. In *Proceedings of the International Conference on Artificial Intelligence and Soft Computing (ICAISC '17)*, Vol. 10246, 31–42.
- [40] Juan Manuel Florez, Oscar Chaparro, Christoph Treude, and Andrian Marcus. 2021. Combining query reduction and expansion for text-retrieval-based bug localization. In *Proceedings of the 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '21)*, 166–176.
- [41] George Forman and Martin Scholz. 2010. Apples-to-apples in cross-validation studies: Pitfalls in classifier performance measurement. *SIGKDD Explorations* 12 (2010), 49–57.
- [42] Yoav Freund and Robert E. Schapire. 1995. A decision-theoretic generalization of on-line learning and an application to boosting. In *Proceedings of the European Conference on Computational Learning Theory*, 23–37.
- [43] Jerome H. Friedman. 2001. Greedy function approximation: A gradient boosting machine. *Annals of Statistics* 29 (2001), 1189–1232.
- [44] Nobukazu Fukuda, Chao Wu, Shingo Horiuchi, and Kenichi Tayama. 2022. Fault report generation for ICT systems by jointly learning time-series and text data. In *Proceedings of the 2022 IEEE/IFIP Network Operations and Management Symposium (NOMS '22)*, 1–9.
- [45] Guofeng Gao, Hui Li, Rong Chen, Xin Ge, and Shikai Guo. 2018. Identification of high priority bug reports via integration method. In *Proceedings of the 6th CCF Conference on Big Data (Big Data '18)*, Vol. 945, 336–349.
- [46] Harold Valdivia Garcia, Emad Shihab, and Meiyappan Nagappan. 2018. Characterizing and predicting blocking bugs in open source projects. *Journal of Systems and Software* 143 (2018), 44–58.
- [47] Marian Gawron, Feng Cheng, and Christoph Meinel. 2017. Automatic vulnerability classification using machine learning. In *Proceedings of the 12th International Conference on Risks and Security of Internet and Systems (CRiSIS '17), Revised Selected Papers*, Vol. 10694, 3–17.
- [48] Xiuting Ge, Chunrong Fang, Meiyuan Qian, Yu Ge, and Mingshuang Qing. 2022. Locality-based security bug report identification via active learning. *Information and Software Technology* 147 (2022), 106899.
- [49] Luiz Alberto Ferreira Gomes, Ricardo da Silva Torres, and Mario Lúcio Côrtes. 2019. Bug report severity level prediction in open source software: A survey and research opportunities. *Information and Software Technology* 115 (2019), 58–78.
- [50] Luiz Alberto Ferreira Gomes, Ricardo da Silva Torres, and Mario Lúcio Côrtes. 2021. On the prediction of long-lived bugs: An analysis and comparative study using FLOSS projects. *Information and Software Technology* 132 (2021), 106508.
- [51] Jingzhi Gong and Tao Chen. 2023. Predicting software performance with divide-and-learn. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*, 858–870. DOI: <https://doi.org/10.1145/3611643.3616334>

- [52] Jingzhi Gong and Tao Chen. 2024. Predicting configuration performance in multiple environments with sequential meta-learning. *Proceedings of ACM Software Engineering* 1, FSE (2024), 359–382. DOI: <https://doi.org/10.1145/3643743>
- [53] Jingzhi Gong and Tao Chen. 2025. Deep configuration performance learning: A systematic survey and taxonomy. *ACM Transactions on Software Engineering and Methodology* 34, 1, Article 25 (2025), 1–62. DOI: <https://doi.org/10.1145/3702986>
- [54] Jingzhi Gong, Tao Chen, and Rami Bahsoon. 2025. Dividable configuration performance learning. *IEEE Transactions on Software Engineering* 51, 1 (2025), 106–134. DOI: <https://doi.org/10.1109/TSE.2024.3491945>
- [55] Shikai Guo, Rong Chen, Miaomiao Wei, Hui Li, and Yaqing Liu. 2018. Ensemble data reduction techniques and multi-RSMOTE via fuzzy integral for bug report classification. *IEEE Access* 6 (2018), 45934–45950.
- [56] Som Gupta and Sanjai Kumar Gupta. 2021. An approach to generate the bug report summaries using two-level feature extraction. *Expert Systems with Applications* 176 (2021), 114816.
- [57] Chetna Gupta, Pedro R. M. Inácio, and Mário M. Freire. 2022. Improving software maintenance with improved bug triaging. *Journal of King Saud University-Computer and Information Sciences* 34 (2022), 8757–8764.
- [58] Marlo Haering, Christoph Stanik, and Walid Maalej. 2021. Automatically matching bug reports with related app reviews. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE '21)*, 970–981.
- [59] Jianjun He, Ling Xu, Yuanrui Fan, Zhou Xu, Meng Yan, and Yan Lei. 2020. Deep learning based valid bug reports determination and explanation. In *Proceedings of the 2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE '20)*, 184–194.
- [60] Jianjun He, Ling Xu, Meng Yan, Xin Xia, and Yan Lei. 2020. Duplicate bug report detection using dual-channel convolutional neural networks. In *Proceedings of the 28th International Conference on Program Comprehension*, 117–127.
- [61] Abram Hindle, Anahita Alipour, and Eleni Stroulia. 2016. A contextual approach towards more accurate duplicate bug report detection and ranking. *Empirical Software Engineering* 21 (2016), 368–410.
- [62] Rin Hirakawa, Keitaro Tominaga, and Yoshihisa Nakatoh. 2020. Study on automatic defect report classification system with self attention visualization. In *Proceedings of the 2020 IEEE International Conference on Consumer Electronics (ICCE)*. IEEE, 1–2.
- [63] Thomas Hirsch and Birgit Hofer. 2022. Using textual bug reports to predict the fault category of software bugs. *Array* 15 (2022), 100189.
- [64] Hao Hu, Hongyu Zhang, Jifeng Xuan, and Weigang Sun. 2014. Effective bug triage based on historical bug-fix information. In *Proceedings of the 2014 IEEE 25th International Symposium on Software Reliability Engineering*, 122–132.
- [65] Zijie Huang, Zhiqing Shao, Guisheng Fan, Huiqun Yu, Kang Yang, and Ziyi Zhou. 2022. Bug report priority prediction using developer-oriented socio-technical features. In *Proceedings of the 13th Asia-Pacific Symposium on Internetware (Internetware '22)*, 202–211.
- [66] Xuan Huo, Ferdian Thung, Ming Li, David Lo, and Shu-Ting Shi. 2021. Deep transfer bug localization. *IEEE Transactions on Software Engineering* 47 (2021), 1368–1380.
- [67] Haruna Isotani, Hironori Washizaki, Yoshiaki Fukazawa, Tsutomu Nomoto, Saori Ouji, and Shinobu Saito. 2021. Duplicate bug report detection by using sentence embedding and fine-tuning. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME '21)*. IEEE, 535–544.
- [68] Hadi Jahanshahi and Mucahit Cevik. 2022. S-DABT: Schedule and dependency-aware bug triage in open-source bug tracking systems. *Information and Software Technology* 151 (2022), 107025.
- [69] Hadi Jahanshahi, Kritika Chhabra, Mucahit Cevik, and Aype Bapar. 2021. DABT: A dependency-aware bug triaging method. In *Proceedings of the 25th International Conference on Evaluation and Assessment in Software Engineering*, 221–230.
- [70] Deepak Kumar Jain, Akshi Kumar, Saurabh Raj Sangwan, Gia Nhu Nguyen, and Prayag Tiwari. 2019. A particle swarm optimized learning model of fault classification in web-apps. *IEEE Access* 7 (2019), 18480–18489.
- [71] Kasthuri Jayarajah, Meera Radhakrishnan, and Camellia Zakaria. 2016. Duplicate issue detection for the Android open source project. In *Proceedings of the 5th International Workshop on Software Mining*, 24–31.
- [72] Tao Ji, Jinkun Pan, Liqian Chen, and Xiaoguang Mao. 2018. Identifying supplementary bug-fix commits. In *Proceedings of the 2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC '18)*, Volume 1, 184–193.
- [73] Yuan Jiang, Pengcheng Lu, Xiaohong Su, and Tiantian Wang. 2020. LTRWES: A new framework for security bug report detection. *Information and Software Technology* 124 (2020), 106314.
- [74] Yuan Jiang, Xiaohong Su, Christoph Treude, Chao Shang, and Tiantian Wang. 2023. Does Deep Learning improve the performance of duplicate bug report detection? An empirical study. *Journal of Systems and Software* 198 (2023), 111607.
- [75] Shubhra Goyal Jindal and Arvinder Kaur. 2020. Automatic keyword and sentence-based text summarization for software bug reports. *IEEE Access* 8 (2020), 65352–65370.

- [76] Leif Jonsson, Markus Borg, David Broman, Kristian Sandahl, Sigrid Eldh, and Per Runeson. 2016. Automated bug assignment: Ensemble-based machine learning in large scale industrial contexts. *Empirical Software Engineering* 21 (2016), 1533–1578.
- [77] Leif Jonsson, David Broman, Måns Magnusson, Kristian Sandahl, Mattias Villani, and Sigrid Eldh. 2016. Automatic localization of bugs to faulty components in large scale software systems using Bayesian classification. In *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS '16)*, 423–430.
- [78] Mahesh V. Joshi. 2002. On evaluating performance of classifiers for rare classes. In *Proceedings of the 2002 IEEE International Conference on Data Mining*, 641–644.
- [79] Rafael Kallis, Andrea Di Sorbo, Gerardo Canfora, and Sebastiano Panichella. 2021. Predicting issue types on GitHub. *Science of Computer Programming* 205 (2021), 102598.
- [80] Nivir Kanti Singha Roy and Bruno Rossi. 2014. Towards an improvement of bug severity classification. In *Proceedings of the 2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications*, 269–276.
- [81] Yutaro Kashiwa, Hayato Yoshiyuki, Yusuke Kukita, and Masao Ohira. 2014. A pilot study of diversity in high impact bugs. In *Proceedings of the 30th IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 536–540.
- [82] Arvinder Kaur and Shubhra Goyal Jindal. 2019. Text analytics based severity prediction of software bugs for Apache projects. *International Journal of System Assurance Engineering and Management* 10 (2019), 765–782.
- [83] Saket Khatiwada, Miroslav Tushev, and Anas Mahmoud. 2020. On combining IR methods to improve bug localization. In *Proceedings of the 28th International Conference on Program Comprehension (ICPC '20)*, 252–262.
- [84] Jungyeon Kim and Geunseok Yang. 2022. Bug severity prediction algorithm using topic-based feature selection and CNN-LSTM algorithm. *IEEE Access* 10 (2022), 94643–94651.
- [85] Taemin Kim and Geunseok Yang. 2022. Predicting duplicate in bug report using topic-based duplicate learning with fine tuning-based BERT algorithm. *IEEE Access* 10 (2022), 129666–129675.
- [86] Kisub Kim, Sankalp Ghatpande, Kui Liu, Anil Koyuncu, Dongsun Kim, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2022. DigBug—Pre/post-processing operator selection for accurate bug localization. *Journal of Systems and Software* 189 (2022), 111300.
- [87] Misoo Kim, Youngkyoung Kim, and Eunseok Lee. 2021. A novel automatic query expansion with word embedding for IR-based bug localization. In *Proceedings of the 2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 276–287.
- [88] Youngkyoung Kim, Misoo Kim, and Eunseok Lee. 2022. Feature assortment for deep learning-based bug localization with a program graph. In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing (SAC '22), Virtual Event*, 1536–1544.
- [89] Barbara A. Kitchenham, Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen G. Linkman. 2009. Systematic literature reviews in software engineering—A systematic literature review. *Information & Software Technology* 51 (2009), 7–15.
- [90] Pavneet Singh Kochhar, Ferdian Thung, and David Lo. 2014. Automatic fine-grained issue report reclassification. In *Proceedings of the 2014 19th International Conference on Engineering of Complex Computer Systems*, 126–135.
- [91] Ashima Kukkar, Rajni Mohana, Yugal Kumar, Anand Nayyar, Muhammad Bilal, and Kyung-Sup Kwak. 2020. Duplicate bug report detection and classification system based on deep learning technique. *IEEE Access* 8 (2020), 200749–200763.
- [92] Lov Kumar, Mukesh Kumar, Lalita Bhanu Murthy Neti, Sanjay Misra, Vipul Kocher, and Srinivas Padmanabhuni. 2021. An empirical study on application of word embedding techniques for prediction of software defect severity level. In *Proceedings of the 16th Conference on Computer Science and Intelligence Systems*, Vol. 25, 477–484.
- [93] An Ngoc Lam, Anh Tuan Nguyen, Hoan Anh Nguyen, and Tien N. Nguyen. 2017. Bug localization with combination of deep learning and information retrieval. In *Proceedings of the 25th International Conference on Program Comprehension (ICPC '17)*, 229.
- [94] Tien-Duy B. Le, Ferdian Thung, and David Lo. 2014. Predicting effectiveness of IR-based bug localization techniques. In *Proceedings of the 2014 IEEE 25th International Symposium on Software Reliability Engineering*, 335–345.
- [95] Tien-Duy B. Le, Ferdian Thung, and David Lo. 2017. Will this localization tool be effective for this bug? Mitigating the impact of unreliability of information retrieval based bug localization tools. *Empirical Software Engineering* 22 (2017), 2237–2279.
- [96] David D. Lewis. 1998. Naive (Bayes) at forty: The independence assumption in information retrieval. In *Proceedings of the 10th European Conference on Machine Learning (ECML '98)*, Vol. 1398, 4–15.
- [97] Yingling Li, Xing Che, Yuekai Huang, Junjie Wang, Song Wang, Yawen Wang, and Qing Wang. 2022. A tale of two tasks: Automated issue priority prediction with deep multi-task learning. In *Proceedings of the 16th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 1–11.
- [98] Miqing Li, Tao Chen, and Xin Yao. 2022. How to evaluate solutions in pareto-based search-based software engineering: A critical review and methodological guidance. *IEEE Transactions on Software Engineering* 48, 5 (2022), 1771–1799. DOI: <https://doi.org/10.1109/TSE.2020.3036108>

- [99] Xiaochen Li, He Jiang, Dong Liu, Zhilei Ren, and Ge Li. 2018. Unsupervised deep bug report summarization. In *Proceedings of the 26th Conference on Program Comprehension (ICPC '18)*, 144–155.
- [100] Xia Li, Wei Li, Yuqun Zhang, and Lingming Zhang. 2019. DeepFL: Integrating multiple fault diagnosis dimensions for deep fault localization. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '19)*, 169–180.
- [101] Zhong Li, Minxue Pan, Yu Pei, Tian Zhang, Linzhang Wang, and Xuandong Li. 2022. DeepLabel: Automated issue classification for issue tracking systems. In *Proceedings of the 13th Asia-Pacific Symposium on Internetware*, 231–241.
- [102] Hongyan Li, Meng Yan, Weifeng Sun, Xiao Liu, and Yunsong Wu. 2023. A first look at bug report templates on GitHub. *Journal of Systems and Software* 202 (2023), 111709.
- [103] Hongyuan Liang, Yue Huang, and Tao Chen. 2025. The same only different: On information modality for configuration performance analysis. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE '25)*. IEEE, 2522–2534. DOI : <https://doi.org/10.1109/ICSE55347.2025.00212>
- [104] Hongliang Liang, Lu Sun, Meilin Wang, and Yuxing Yang. 2019. Deep learning with customized abstract syntax tree for bug localization. *IEEE Access* 7 (2019), 116309–116320.
- [105] Meng-Jie Lin, Cheng-Zen Yang, Chao-Yuan Lee, and Chun-Chang Chen. 2016. Enhancements for duplication detection in bug reports with manifold correlation features. *Journal of Systems and Software* 121 (2016), 223–233.
- [106] Guoming Long and Tao Chen. 2022. On reporting performance and accuracy bugs for deep learning frameworks: An exploratory study from GitHub. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering 2022 (EASE '22)*. Mirosław Staron, Christian Berger, Jocelyn Simmonds, and Rafael Prikladnicki (Eds.), ACM, 90–99. DOI : <https://doi.org/10.1145/3530019.3530029>
- [107] Guoming Long, Tao Chen, and Georgina Cosma. 2022. Multifaceted hierarchical report identification for non-functional bugs in deep learning frameworks. In *Proceedings of the 29th Asia-Pacific Software Engineering Conference (APSEC '22)*, Virtual Event, 289–298.
- [108] Zhengmao Luo, Wenyao Wang, and Cen Caichun. 2023. Improving bug localization with effective contrastive learning representation. *IEEE Access* 11 (2023), 32523–32533.
- [109] Youpeng Ma, Tao Chen, and Ke Li. 2025. Faster configuration performance bug testing with neural dual-level prioritization. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE '25)*. IEEE, 988–1000. DOI : <https://doi.org/10.1109/ICSE55347.2025.00201>
- [110] Xiaoxue Ma, Jacky Keung, Zhen Yang, Xiao Yu, Yishu Li, and Hao Zhang. 2022. CASMS: Combining clustering with attention semantic model for identifying security bug reports. *Information and Software Technology* 147 (2022), 106906.
- [111] Xiaoxue Ma, Jacky Wai Keung, Xiao Yu, Huiqi Zou, Jingyu Zhang, and Yishu Li. 2023. AttSum: A deep attention-based summarization model for bug report title generation. *IEEE Transactions on Reliability* 72 (2023), 1663–1677.
- [112] Senthil Mani, Anush Sankaran, and Rahul Aralikatte. 2019. DeepTriage: Exploring the effectiveness of deep learning for bug triaging. In *Proceedings of the ACM India Joint International Conference on Data Science and Management of Data (COMAD/CODS '19)*, 171–179.
- [113] Montassar Ben Messaoud, Asma Miladi, Ilyes Jenhani, Mohamed Wiem Mkaouer, and Lobna Ghadhab. 2023. Duplicate bug report detection using an attention-based neural language model. *IEEE Transactions on Reliability* 72 (2023), 846–858.
- [114] Tomás Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. In *Proceedings of the 1st International Conference on Learning Representations (ICLR '13)*, Workshop Track Proceedings.
- [115] Nima Miryeganeh, Sepehr Hashtroudi, and Hadi Hemmati. 2021. GloBug: Using global data in Fault Localization. *Journal of Systems and Software* 177 (2021), 110961.
- [116] Amr Mansour Mohsen, Hesham A. Hassan, Khaled Wassif, Ramadan Moawad, and Soha Makady. 2023. Enhancing bug localization using phase-based approach. *IEEE Access* 11 (2023), 35901–35913.
- [117] Hufsa Mohsin and Chongyang Shi. 2021. SPBC: A self-paced learning model for bug classification from historical repositories of open-source software. *Expert Systems with Applications* 167 (2021), 113808.
- [118] Hufsa Mohsin, Chongyang Shi, Shufeng Hao, and He Jiang. 2022. SPAN: A self-paced association augmentation and node embedding-based model for software bug classification and assignment. *Knowledge-Based Systems* 236 (2022), 107711.
- [119] Behzad Soleimani Neysiani, Seyed Morteza Babamir, and Masayoshi Aritsugi. 2020. Efficient feature extraction model for validation performance improvement of duplicate bug report detection in software bug triage systems. *Information and Software Technology* 126 (2020), 106344.
- [120] Masao Ohira, Yutaro Kashiwa, Yosuke Yamatani, Hayato Yoshiyuki, Yoshiya Maeda, Nachai Limsettho, Keisuke Fujino, Hideaki Hata, Akinori Ihara, and Kenichi Matsumoto. 2015. A dataset of high impact bugs: Manually-classified issue reports. In *Proceedings of the 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*, 518–521.

- [121] Nitish Pandey, Debarshi Kumar Sanyal, Abir Hudait, and Amitava Sen. 2017. Automated classification of software issue reports using machine learning techniques: An empirical study. *Innovations in Systems and Software Engineering* 13 (2017), 279–297.
- [122] Sebastiano Panichella, Gerardo Canfora, and Andrea Di Sorbo. 2021. “Won’t We Fix this Issue?” Qualitative characterization and automated identification of wontfix issues on GitHub. *Information and Software Technology* 139 (2021), 106665.
- [123] Sunnyeo Park, Dohyeok Kim, and Soeul Son. 2019. An empirical study of prioritizing JavaScript engine crashes via machine learning. In *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security (AsiaCCS ’19)*, 646–657.
- [124] Payola Peters, Thein Than Tun, Yijun Yu, and Bashar Nuseibeh. 2019. Text filtering and ranking for security bug report prediction. *IEEE Transactions on Software Engineering* 45 (2019), 615–631.
- [125] Sravya Polisetty, Andriy Miranskyy, and Ayşe Başar. 2019. On usefulness of the deep-learning-based bug localization models to practitioners. In *Proceedings of the 15th International Conference on Predictive Models and Data Analytics in Software Engineering*, 16–25.
- [126] Waheed Yousuf Ramay, Qasim Umer, Xu-Cheng Yin, Chao Zhu, and Inam Illahi. 2019. Deep neural network-based severity prediction of bug reports. *IEEE Access* 7 (2019), 46846–46857.
- [127] Shivani Rao, Henry Medeiros, and Avinash Kak. 2015. Comparing incremental latent semantic analysis algorithms for efficient retrieval from software libraries for bug localization. *ACM SIGSOFT Software Engineering Notes* 40, 1 (2015), 1–8.
- [128] Abdul Razzaq, Jim Buckley, James Vincent Patten, Muslim Chochlov, and Ashish Rajendra Sai. 2021. BoostNSift: A query boosting and code sifting technique for method level bug localization. In *Proceedings of the 21st IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM ’21)*, 81–91.
- [129] Irina Rish. 2001. An empirical study of the naive Bayes classifier. In *Proceedings of the IJCAI 2001 Workshop on Empirical Methods in Artificial Intelligence*, Vol. 3, 41–46.
- [130] Thiago Marques Rocha and André Luiz da Costa Carvalho. 2021. SiameseQAT: A semantic context-based duplicate bug report detection using replicated cluster information. *IEEE Access* 9 (2021), 44610–44630.
- [131] Irving Muller Rodrigues, Daniel Aloise, Eraldo Rezende Fernandes, and Michel R. Dagenais. 2020. A soft alignment model for bug deduplication. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR ’20)*, 43–53.
- [132] Korosh Koochekian Sabor, Mohammad Hamdaqa, and Abdelwahab Hamou-Lhadj. 2016. Automatic prediction of the severity of bugs using stack traces. In *Proceedings of the 26th Annual International Conference on Computer Science and Software Engineering (CASCON ’16)*. Marcellus Mindel, Blake Jones, Hausi A. Müller, and Vio Onut (Eds.), IBM/ACM, 96–105. Retrieved from <http://dl.acm.org/citation.cfm?id=3049887>
- [133] Korosh Koochekian Sabor, Mohammad Hamdaqa, and Abdelwahab Hamou-Lhadj. 2020. Automatic prediction of the severity of bugs using stack traces and categorical features. *Information and Software Technology* 123 (2020), 106205.
- [134] Reza Sepahvand, Reza Akbari, Behnaz Jamasb, Sattar Hashemi, and Omid Boushehrian. 2023. Using word embedding and convolution neural network for bug triaging by considering design flaws. *Science of Computer Programming* 228 (2023), 102945.
- [135] Meera Sharma, Madhu Kumari, Rajeev Kumar Singh, and V. B. Singh. 2014. Multiattribute based machine learning models for severity prediction in cross project context. In *Proceedings of the International Conference on Computational Science and Its Applications (ICCSA ’14)*.
- [136] Gitika Sharma, Sumit Sharma, and Shruti Gujral. 2015. A novel way of assessing software bug severity using dictionary of critical terms. *Procedia Computer Science* 70 (2015), 632–639.
- [137] Zhendong Shi, Jacky Keung, Kwabena Ebo Bennin, and Xingjun Zhang. 2018. Comparing learning to rank techniques in hybrid bug localization. *Applied Soft Computing* 62 (2018), 636–648.
- [138] V. B. Singh, Sanjay Misra, and Meera Sharma. 2017. Bug severity assessment in cross project context and identifying training candidates. *Journal of Information & Knowledge Management* 16, Article 1750005 (2017), 1–30.
- [139] Jonathan D. Strate and Phillip A. Laplante. 2013. A literature review of research in software defect reporting. *IEEE Transactions on Reliability* 62 (2013), 444–454.
- [140] Yu Su, Xinpeng Hu, Xiang Chen, Yubin Qu, and Qianshuang Meng. 2022. CIL-BSP: Bug report severity prediction based on class imbalanced learning. In *Proceedings of the 22nd IEEE International Conference on Software Quality, Reliability and Security (QRS ’22)—Companion*, 298–306.
- [141] Hongyu Sun, Guoliang Ou, Ziqiu Zheng, Lei Liao, He Wang, and Yuqing Zhang. 2023. Inconsistent measurement and incorrect detection of software names in security vulnerability reports. *Computers & Security* 135 (2023), 103477.
- [142] Youshuai Tan, Sijie Xu, Zhaowei Wang, Tao Zhang, Zhou Xu, and Xiapu Luo. 2020. Bug severity prediction using question-and-answer pairs from Stack Overflow. *Journal of Systems and Software* 165 (2020), 110567.

- [143] M. Irtaza Nawaz Tarar, Mubashir Ali, and Wasi Haider Butt. 2019. Bug report summarization: A systematic literature review. In *Proceedings of the 11th International Conference on Education Technology and Computers (ICETC '19)*, 257–261.
- [144] Ferdian Thung, Xuan-Bach Dinh Le, and David Lo. 2015. Active semi-supervised defect categorization. In *Proceedings of the 2015 IEEE 23rd International Conference on Program Comprehension (ICPC '15)*, 60–70.
- [145] Yuan Tian, David Lo, Xin Xia, and Chengnian Sun. 2015. Automated prediction of bug report priority using multi-factor analysis. *Empirical Software Engineering* 20 (2015), 1354–1383.
- [146] Yuan Tian, Dinusha Wijedasa, David Lo, and Claire Le Goues. 2016. Learning to rank for bug report assignee recommendation. In *Proceedings of the 24th IEEE International Conference on Program Comprehension (ICPC '16)*, 1–10.
- [147] Yao Tong and Xiaofang Zhang. 2021. Crowdsourced test report prioritization considering bug severity. *Information and Software Technology* 139 (2021), 106668.
- [148] Akihiro Tsuruda, Yuki Manabe, and Masayoshi Aritsugi. 2015. Can we detect bug report duplication with unfinished bug reports? In *Proceedings of the 2015 Asia-Pacific Software Engineering Conference (APSEC '15)*, 151–158.
- [149] Jamal Uddin, Rozaida Ghazali, Mustafa Mat Deris, Rashid Naseem, and Habib Shah. 2017. A survey on bug prioritization. *Artificial Intelligence Review* 47 (2017), 145–180.
- [150] Qasim Umer, Hui Liu, and Inam Illahi. 2020. CNN-based automatic prioritization of bug reports. *IEEE Transactions on Reliability* 69 (2020), 1341–1354.
- [151] Qasim Umer, Hui Liu, and Yasir Sultan. 2018. Emotion based automated priority prediction for bug reports. *IEEE Access* 6 (2018), 35743–35752.
- [152] Dae-Sung Wang and Chan-Gun Lee. 2022. Automatic component prediction for issue reports using fine-tuned pretrained language models. *IEEE Access* 10 (2022), 131456–131468.
- [153] Hongbing Wang and Qi Li. 2021. Effective bug triage based on a hybrid neural network. In *Proceedings of the 28th Asia-Pacific Software Engineering Conference (APSEC '21)*, 82–91.
- [154] Junjie Wang, Qiang Cui, Qing Wang, and Song Wang. 2016. Towards effectively test report classification to assist crowdsourced testing. In *Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '16)*.
- [155] Junjie Wang, Qiang Cui, Song Wang, and Qing Wang. 2017. Domain adaptation for test report classification in crowdsourced testing. In *Proceedings of the 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP '17)*.
- [156] Wenyaoyao Wang, Chen-Hao Wu, and Jie He. 2023. CLBPI: Contrastive learning for bug priority inference. *Information and Software Technology* 164 (2023), 107302.
- [157] Xiaojuan Wang, Wenyu Zhang, Shanyan Lai, Chunyang Ye, and Hui Zhou. 2022. The use of pretrained model for matching app reviews and bug reports. In *Proceedings of the 22nd IEEE International Conference on Software Quality, Reliability and Security (QRS '22)*, 242–251.
- [158] Song Wang, Wen Zhang, and Qing Wang. 2014. FixerCache: Unsupervised caching active developers for diverse bug triage. In *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '14)*.
- [159] Ye Wei, Chunfu Zhang, and Teng Ren. 2023. Improving bug severity prediction with domain-specific representation learning. *IEEE Access* 11 (2023), 62829–62839.
- [160] Wei Wen, Tingting Yu, and Jane Huffman Hayes. 2016. CoLUA: Automatically predicting configuration bug reports and extracting configuration options. In *Proceedings of the 27th IEEE International Symposium on Software Reliability Engineering (ISSRE '16)*, 150–161.
- [161] Hongrun Wu, Haiyang Liu, and Yutao Ma. 2018. Empirical study on developer factors affecting tossing path length of bug reports. *IET Software* 12 (2018), 258–270.
- [162] Hongrun Wu, Yutao Ma, Zhenglong Xiang, Chen Yang, and Keqing He. 2022. A spatial-temporal graph neural network framework for automated software bug triaging. *Knowledge-Based Systems* 241 (2022), 108308.
- [163] Xiaoxue Wu, Wei Zheng, Xiang Chen, Fang Wang, and Dejun Mu. 2020. CVE-assisted large-scale security bug report dataset construction method. *Journal of Systems and Software* 160 (2020), 110456.
- [164] Xiaoxue Wu, Wei Zheng, Xiang Chen, Yu Zhao, Tingting Yu, and Dejun Mu. 2021. Improving high-impact bug report prediction with combination of interactive machine learning and active learning. *Information and Software Technology* 133 (2021), 106530.
- [165] Xiaoxue Wu, Wei Zheng, Xin Xia, and David Lo. 2022. Data quality matters: A case study on data label correctness for security bug report prediction. *IEEE Transactions on Software Engineering* 48 (2022), 2541–2556.
- [166] Shengqu Xi, Yuan Yao, Xusheng Xiao, Feng Xu, and Jian Lu. 2018. An effective approach for routing the bug reports to the right fixers. In *Proceedings of the 10th Asia-Pacific Symposium on Internetware (Internetware '18)*, Article 11, 1–10.

- [167] Xin Xia, David Lo, Weiwei Qiu, Xingen Wang, and Bo Zhou. 2014. Automated configuration bug report prediction using text mining. In *Proceedings of the IEEE 38th Annual Computer Software and Applications Conference (COMPSAC '14)*, 107–116.
- [168] Xin Xia, David Lo, Emad Shihab, and Xinyu Wang. 2016. Automated bug report field reassignment and refinement prediction. *IEEE Transactions on Reliability* 65 (2016), 1094–1113.
- [169] Xin Xia, David Lo, Emad Shihab, Xinyu Wang, and Xiaohu Yang. 2015. ELBlocker: Predicting blocking bugs with ensemble imbalance learning. *Information and Software Technology* 61 (2015), 93–106.
- [170] Xin Xia, David Lo, Emad Shihab, Xinyu Wang, and Bo Zhou. 2015. Automatic, high accuracy prediction of reopened bugs. *Automated Software Engineering* 22 (2015), 75–109.
- [171] Xin Xia, David Lo, Xinyu Wang, and Bo Zhou. 2015. Dual analysis for recommending developers to resolve bugs. *Journal of Software: Evolution and Process* 27 (2015), 195–220.
- [172] Zezhen Xiang, Jingzhi Gong, and Tao Chen. 2026. Dually hierarchical drift adaptation for online configuration performance learning. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM.
- [173] Yan Xiao, Jacky Keung, Qing Mi, and Kwabena E. Bennin. 2017. Improving bug localization with an enhanced convolutional neural network. In *Proceedings of the 2017 24th Asia-Pacific Software Engineering Conference (APSEC)*, 338–347.
- [174] Yan Xiao, Jacky Keung, Qing Mi, and Kwabena E. Bennin. 2018. Bug localization with semantic and structural features using convolutional neural network and cascade forest. In *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018*, 101–111.
- [175] Bowen Xu, David Lo, Xin Xia, Ashish Sureka, and Shanping Li. 2015. EFSPredictor: Predicting configuration bugs with ensemble feature selection. In *Proceedings of the 2015 Asia-Pacific Software Engineering Conference (APSEC '15)*, 206–213.
- [176] Jifeng Xuan, He Jiang, Yan Hu, Zhilei Ren, Weiqin Zou, Zhongxuan Luo, and Xindong Wu. 2014. Towards effective bug triage with software data reduction techniques. *IEEE Transactions on Knowledge and Data Engineering* 27, 1 (2014), 264–280.
- [177] Geunseok Yang, Seungsuk Baek, Jung-Won Lee, and Byungjeong Lee. 2017. Analyzing emotion words to predict severity of software bugs: A case study of open source projects. In *Proceedings of the Symposium on Applied Computing (SAC '17)*, 1280–1287.
- [178] Xin-Li Yang, David Lo, Xin Xia, Qiao Huang, and Jian-Ling Sun. 2017. High-impact bug report identification with imbalanced learning strategies. *Journal of Computer Science and Technology* 32 (2017), 181–198.
- [179] Geunseok Yang, Kyeongsic Min, Jung-Won Lee, and Byungjeong Lee. 2019. Applying topic modeling and similarity for predicting bug severity in cross projects. *KSII Transactions on Internet and Information Systems* 13 (2019), 1583–1598.
- [180] Geunseok Yang, Tao Zhang, and Byungjeong Lee. 2014. Towards semi-automatic bug triage and severity prediction based on topic model and multi-feature of bug reports. In *Proceedings of the IEEE 38th Annual Computer Software and Applications Conference (COMPSAC '14)*, 97–106.
- [181] Yulong Ye, Tao Chen, and Miqing Li. 2025. Distilled lifelong self-adaptation for configurable systems. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE '25)*. IEEE, 1333–1345. DOI: <https://doi.org/10.1109/ICSE55347.2025.00094>
- [182] Xin Ye, Fan Fang, John Wu, Razvan C. Bunescu, and Chang Liu. 2018. Bug report classification using LSTM architecture for more accurate software defect locating. In *Proceedings of the 17th IEEE International Conference on Machine Learning and Applications (ICMLA '18)*, 1438–1445.
- [183] Syed Farhan Alam Zaidi, Faraz Malik Awan, Minsoo Lee, Honguk Woo, and Chan-Gun Lee. 2020. Applying convolutional neural networks with different word representation techniques to recommend bug fixers. *IEEE Access* 8 (2020), 213729–213747.
- [184] Motahareh Bahrani Zanjani, Huzefa H. Kagdi, and Christian Bird. 2015. Using developer-interaction trails to triage change requests. In *Proceedings of the 12th IEEE/ACM Working Conference on Mining Software Repositories (MSR '15)*, 88–98.
- [185] R. Zhang and J. Ma. 2008. An improved SVM method P-SVM for classification of remotely sensed data. *International Journal of Remote Sensing* 29 (2008), 6029–6036.
- [186] Tao Zhang, Jiachi Chen, He Jiang, Xiapu Luo, and Xin Xia. 2017. Bug report enrichment with application of automated fixer recommendation. In *Proceedings of the 25th International Conference on Program Comprehension (ICPC '17)*, 230–240.
- [187] Tao Zhang, Jiachi Chen, Xiapu Luo, and Tao Li. 2019. Bug reports for desktop software and mobile apps in GitHub: What's the difference? *IEEE Software* 36, 1 (2019), 63–71. DOI: <https://doi.org/10.1109/MS.2017.377142400>
- [188] Tao Zhang, Jiachi Chen, Geunseok Yang, Byungjeong Lee, and Xiapu Luo. 2016. Towards more accurate severity prediction and fixer recommendation of software bugs. *Journal of Systems and Software* 117 (2016), 166–184.

- [189] Ting Zhang, DongGyun Han, Venkatesh Vinayakarao, Ivana Clairine Irsan, Bowen Xu, Ferdian Thung, David Lo, and Lingxiao Jiang. 2023. Duplicate bug report detection: How far are we? *IEEE Transactions on Software Engineering and Methodology* 32, Article 97 (2023), 1–32.
- [190] Jie Zhang, XiaoYin Wang, Dan Hao, Bing Xie, Lu Zhang, and Hong Mei. 2015. A survey on bug-report analysis. *Science China Information Sciences* 58 (2015), 1–24.
- [191] Wen Zhang, Song Wang, and Qing Wang. 2016. KSAP: An approach to bug report assignment using KNN search and heterogeneous proximity. *Information and Software Technology* 70 (2016), 68–84.
- [192] Jinglei Zhang, Rui Xie, Wei Ye, Yuhan Zhang, and Shikun Zhang. 2020. Exploiting code knowledge graph for bug localization via bi-directional attention. In *Proceedings of the 28th International Conference on Program Comprehension (ICPC '20)*, 219–229.
- [193] Wen Zhang, Jiangpeng Zhao, and Song Wang. 2021. SusTriage: Sustainable bug triage with multi-modal ensemble learning. In *Proceedings of the IEEE/WIC/ACM International Conference on Web Intelligence and Intelligent Agent Technology*, 441–448.
- [194] Wei Zheng, Zheng Chen, Xiaoxue Wu, Weiqiang Fu, Bowen Sun, and Jingyuan Cheng. 2021. A domain knowledge-guided lightweight approach for security bug reports prediction. In *Proceedings of the 2021 8th International Conference on Dependable Systems and Their Applications (DSA '21)*, 359–368.
- [195] Wei Zheng, Manqing Zhang, Hui Tang, Yuanfang Cai, Xiang Chen, Xiaoxue Wu, and Abubakar Omari Abdallah Semasaba. 2021. Automatically identifying bug reports with tactical vulnerabilities by deep feature learning. In *Proceedings of the 2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE '21)*, 333–344.
- [196] Cheng Zhou, Bin Li, and Xiaobing Sun. 2020. Improving software bug-specific named entity recognition with deep neural network. *Journal of Systems and Software* 165 (2020), 110572.
- [197] Bo Zhou, Iulian Neamtii, and Rajiv Gupta. 2015. Predicting concurrency bugs: How many, what kind and where are they? In *Proceedings of the 19th International Conference on Evaluation and Assessment in Software Engineering (EASE '15)*, Article 6, 1–10.
- [198] Ziye Zhu, Hanghang Tong, Yu Wang, and Yun Li. 2022. Enhancing bug localization with bug report decomposition and code hierarchical network. *Knowledge-Based Systems* 248 (2022), 108741.
- [199] Weiqin Zou, David Lo, Zhenyu Chen, Xin Xia, Yang Feng, and Baowen Xu. 2018. How practitioners perceive automated bug report management techniques. *IEEE Transactions on Software Engineering* 46, 8 (2018), 836–862.
- [200] Jie Zou, Ling Xu, Mengning Yang, Xiaohong Zhang, Jun Zeng, and Sachio Hirokawa. 2016. Automated duplicate bug report detection using multi-factor analysis. *IEICE Transactions on Information and Systems* 99-D (2016), 1762–1775.

Received 5 October 2024; revised 26 March 2025; accepted 1 July 2025